Linux Log File Analysis, Automation, and SIEM Visualization

A hands-on SOC-focused lab: manually triaging Linux authentication logs, automating detection with Python, and ingesting, querying, and visualising the same dataset in Splunk Enterprise.

Tanmay Sarkar · SOC Analyst Lab Project · Dataset: LogHub Linux_2k.log · SIEM: Splunk Enterprise 10.4.1

Objective

The objective of this project is to build the core workflow of a Tier 1 SOC analyst — monitor, detect, analyse — across three levels of tooling maturity, using a single real-world Linux authentication log. The lab begins with manual review in a text editor and a spreadsheet, progresses to a Python script that classifies suspicious events automatically and exports them to CSV, and finishes by ingesting the same file into Splunk Enterprise to query, correlate, and visualise the activity at scale. The intent is to demonstrate not only that each tool works, but why an analyst moves from one to the next as data volume grows beyond what manual review can cover.

Lab Environment

Component	Name	Configuration
Dataset	Linux_2k.log	LogHub Linux authentication log — 2,000 lines spanning 14 June to 27 July
Monitored host	combo	Linux server producing the sshd, ftpd, su, and logrotate events analysed
Manual analysis	Microsoft Excel	Linux_2k.xlsx — all 2,000 lines parsed into 8 structured columns
Automation	Python 3.x in Visual Studio Code	Log_Analysis_Automation.py (detection), log_to_excel.py (parsing)
SIEM	Splunk Enterprise 10.4.1	Local single-instance install on Windows, web UI at 127.0.0.1:8000
Splunk data input	source="Linux_2k.log" host="tsarkar1" sourcetype="Linux"	1,296 events indexed into the default index
Outputs	suspicious_logs.csv Full_suspicious_logs.csv	140 entries (lines 200–500) and 607 entries (full file)

Prerequisites and Tools

- Basic knowledge of Linux authentication, SSH, and the syslog message format
- Visual Studio Code (or any text editor) for reviewing the raw log file
- Microsoft Excel or Google Sheets for tabulating findings
- Python 3.x installed and available on the command line
- Splunk Enterprise (free 60-day trial) for the SIEM stage

- The Linux_2k.log dataset from the LogHub repository

Scope Note

This walkthrough documents every analysis step in full. Two categories of screenshot have been deliberately excluded: the Splunk account registration and sign-in pages, and the local Splunk Enterprise login screen. Those screens contain account identifiers and credential fields that should not be published. Every step that produces analytical evidence is documented completely.

Objective 1 — Manual Log Analysis

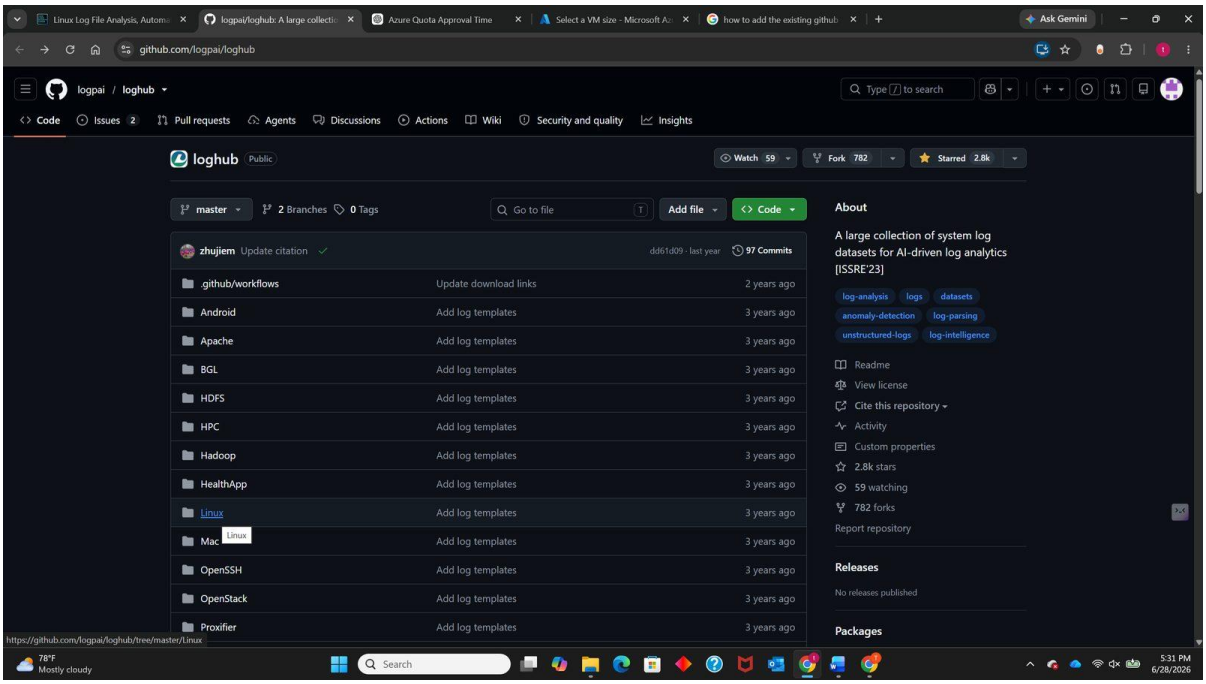
This first stage establishes the baseline: reading a real Linux authentication log by eye, recognising what suspicious activity looks like in raw syslog format, and organising the findings into a structure that can be filtered and counted. Every automated technique used later in this project exists to scale what is done manually here, so the patterns have to be understood before they can be codified.

By the end of this objective the goal is to understand what system logs contain and why they matter, identify failed login attempts in a real-world log file, and practise the basic SOC workflow of monitor → detect → analyse.

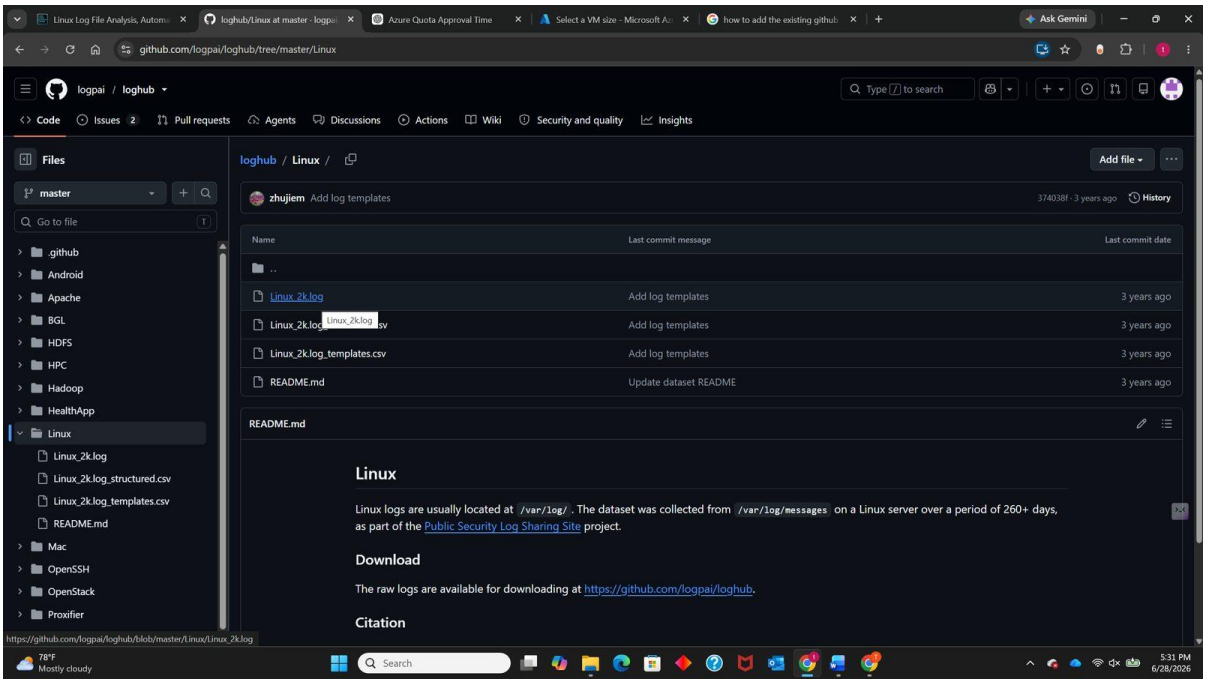
Step 1: Download a Sample Log File

The dataset used throughout all three objectives is Linux_2k.log from the LogHub repository, a curated collection of real-world system log datasets published for log-analysis research. It contains genuine authentication attempts, user sessions, and system events from a Linux host named combo, which makes it far more useful than synthetic data — the noise, the inconsistent formatting, and the irregular attack timing are all real.

1. Open the LogHub repository at https://github.com/logpai/loghub
2. Navigate to the Linux dataset folder
3. Download Linux_2k.log (2,000 lines, approximately 212 KB)



The LogHub repository on GitHub, listing the available log datasets.

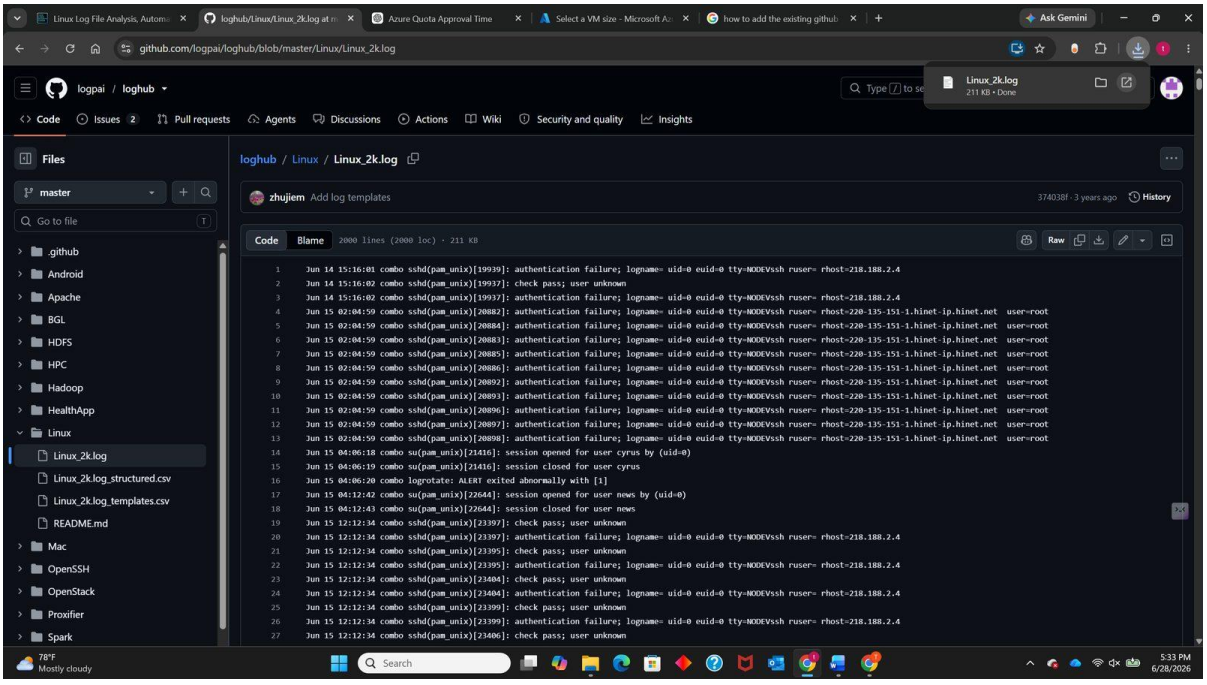


The Linux dataset folder containing Linux_2k.log.

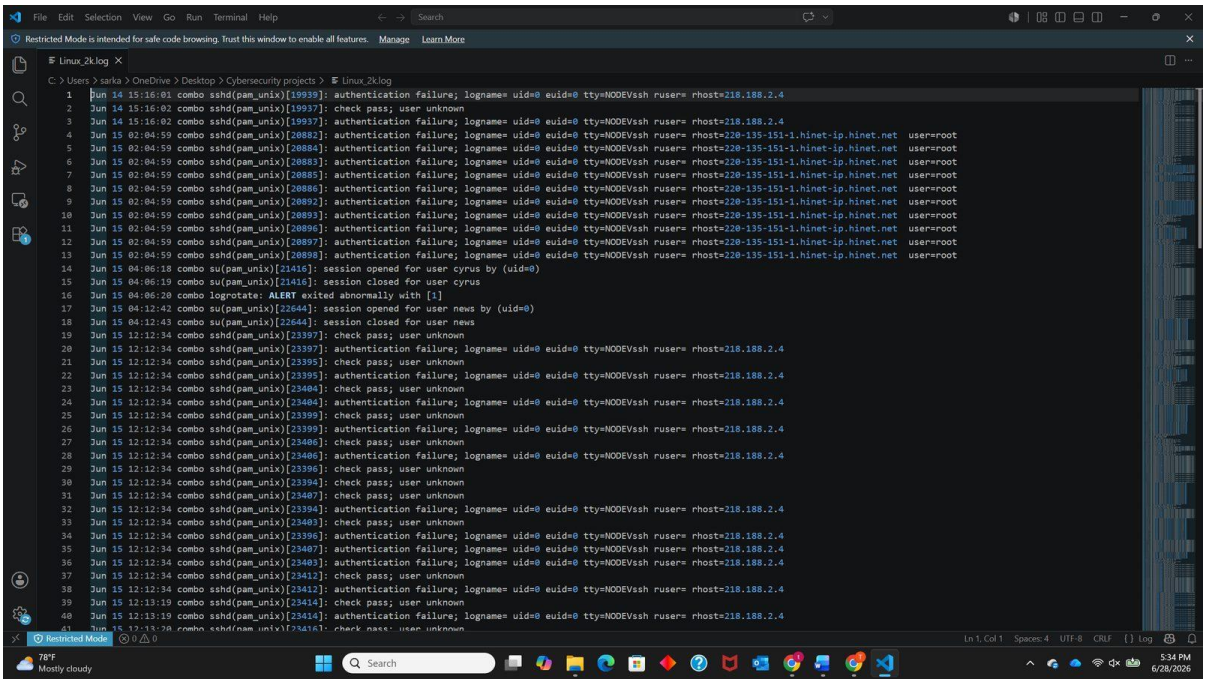
Step 2: Open and Review the Logs

The file was opened in Visual Studio Code. To keep the initial review manageable, analysis was limited to the first 20 to 40 lines rather than attempting to read all 2,000 at once — the same approach an analyst takes when triaging an unfamiliar log source, establishing what normal looks like before hunting for anomalies.

Each line follows the standard syslog structure: timestamp, hostname, service and process ID, then the message body. Recognising this structure is what makes the field extraction in Objectives 2 and 3 possible.



Linux_2k.log opened in Visual Studio Code.



The first 40 lines under review, with authentication failures visible.

Step 3: Identify Suspicious Events

Three categories of suspicious activity were targeted during the review:

- Repeated failed logins from the same source address — the signature of a brute-force attempt
- Unknown or invalid users attempting to authenticate — indicating username enumeration
- System alerts or abnormal service exits — potential signs of an underlying fault or tampering

Within the first 40 lines alone, this review surfaced 23 authentication failures, 12 unknown-user probes, one abnormal service exit (logrotate: ALERT exited abnormally with [1]), and two legitimate session openings for comparison. Two source addresses accounted for all of the failures: 218.188.2.4 with 13 attempts and 220-135-151-1.hinet-ip.hinet.net with 10.

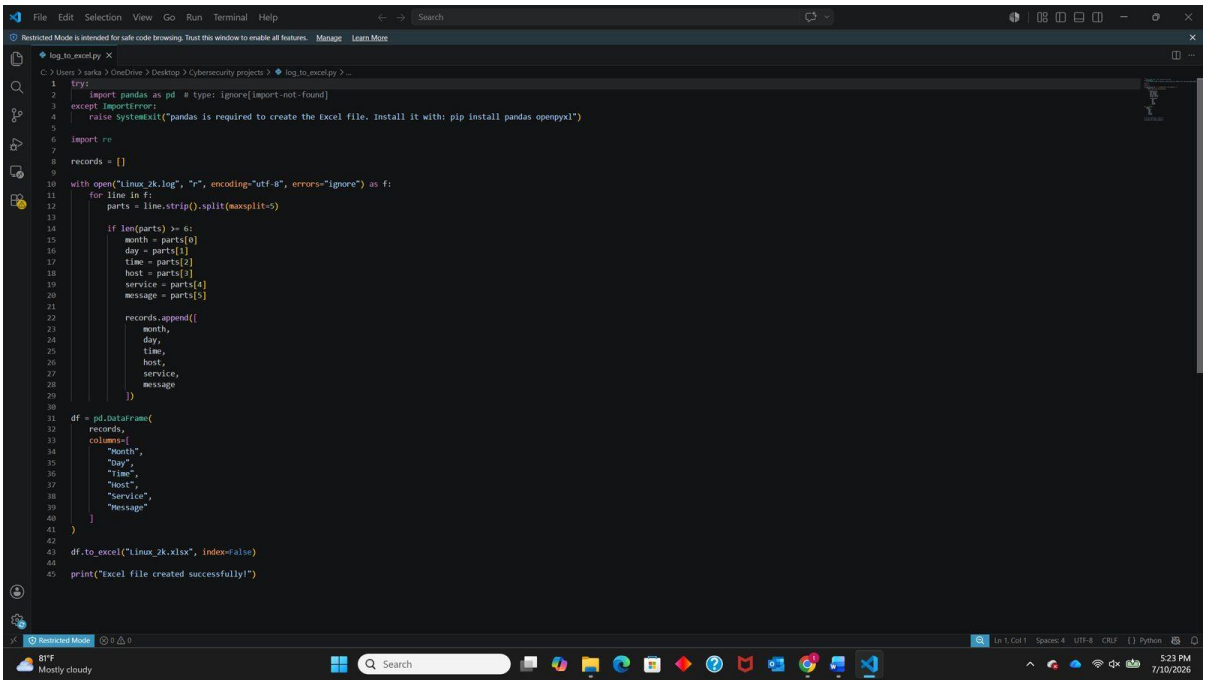
Step 4: Organise Findings

Reading lines in a text editor establishes the pattern but does not scale, and it cannot answer questions such as which source is most active or which account is most targeted. To make the data countable, the full log was parsed into structured spreadsheet columns using a short Python script, log_to_excel.py, which splits each syslog line into its component fields.

```
import pandas as pd

records = []
with open("Linux_2k.log", "r", encoding="utf-8", errors="ignore") as f:
    for line in f:
        parts = line.strip().split(maxsplit=5)
        if len(parts) >= 6:
            records.append(parts[:6])   # Month, Day, Time, Host, Service, Message

df = pd.DataFrame(records,
```

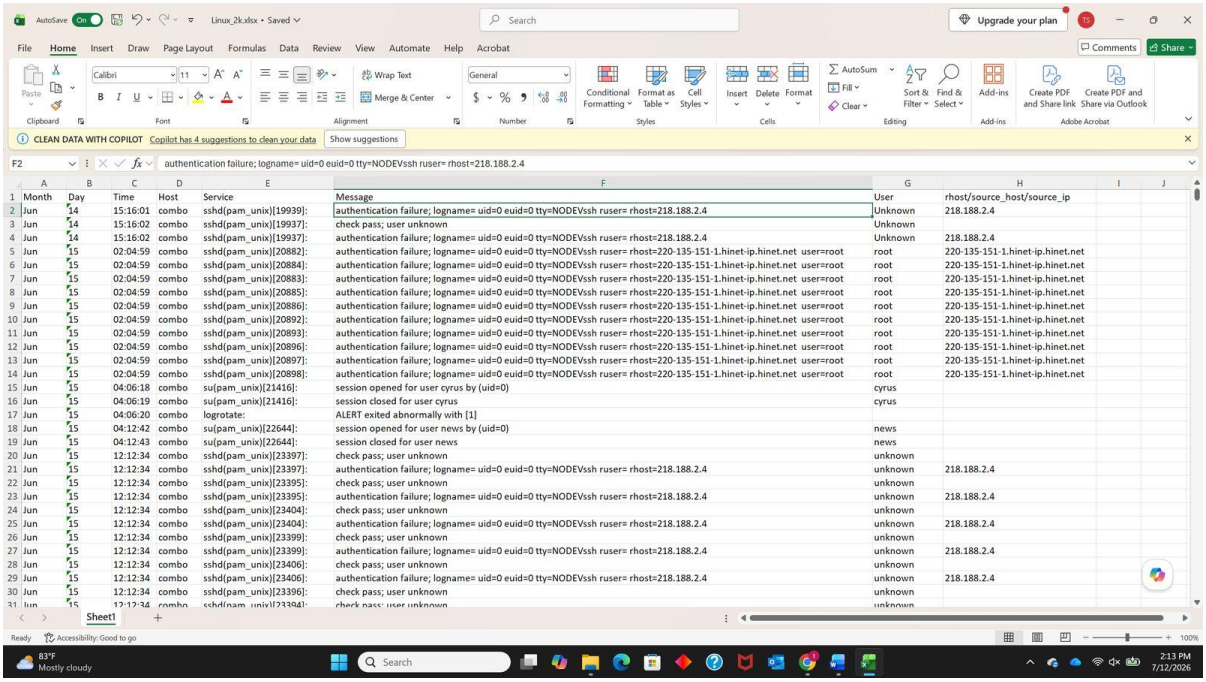
```
        columns=["Month", "Day", "Time", "Host", "Service", "Message"])
df.to_excel("Linux_2k.xlsx", index=False)
```

Two further columns — User and rhost/source_host/source_ip — were added in Excel to isolate the targeted account and the attacking source, producing an eight-column table across all 2,000 rows. With the data in this form, sorting and filtering immediately reveals which addresses repeat and which accounts they target.



```python
try:
    import pandas as pd  # type: ignore[import-not-found]
except ImportError:
    raise SystemExit("pandas is required to create the Excel file. Install it with: pip install pandas openpyxl")

import re

records = []

with open("Linux_2k.log", "r", encoding="utf-8", errors="ignore") as f:
    for line in f:
        parts = line.strip().split(maxsplit=5)

        if len(parts) >= 6:
            month = parts[0]
            day = parts[1]
            time = parts[2]
            host = parts[3]
            service = parts[4]
            message = parts[5]

            records.append([
                month,
                day,
                time,
                host,
                service,
                message
            ])

df = pd.DataFrame(
    records,
    columns=[
        "Month",
        "Day",
        "Time",
        "Host",
        "Service",
        "Message"
    ]
)

df.to_excel("Linux_2k.xlsx", index=False)

print("Excel file created successfully!")
```

log_to_excel.py — parsing the raw syslog lines into structured columns.



Month	Day	Time	Host	Service	Message	User	rhost/source_host/source_ip
Jun	14	15:16:01	combo	sshd(pam_unix)[19939]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=218.188.2.4	Unknown	218.188.2.4
Jun	14	15:16:02	combo	sshd(pam_unix)[19937]:	check pass; user unknown	Unknown	
Jun	14	15:16:02	combo	sshd(pam_unix)[19937]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=218.188.2.4	Unknown	218.188.2.4
Jun	15	02:04:59	combo	sshd(pam_unix)[20882]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=220-135-151-1.hinet-ip.hinet.net user=root	root	220-135-151-1.hinet-ip.hinet.net
Jun	15	02:04:59	combo	sshd(pam_unix)[20884]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=220-135-151-1.hinet-ip.hinet.net user=root	root	220-135-151-1.hinet-ip.hinet.net
Jun	15	02:04:59	combo	sshd(pam_unix)[20883]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=220-135-151-1.hinet-ip.hinet.net user=root	root	220-135-151-1.hinet-ip.hinet.net
Jun	15	02:04:59	combo	sshd(pam_unix)[20885]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=220-135-151-1.hinet-ip.hinet.net user=root	root	220-135-151-1.hinet-ip.hinet.net
Jun	15	02:04:59	combo	sshd(pam_unix)[20886]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=220-135-151-1.hinet-ip.hinet.net user=root	root	220-135-151-1.hinet-ip.hinet.net
Jun	15	02:04:59	combo	sshd(pam_unix)[20892]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=220-135-151-1.hinet-ip.hinet.net user=root	root	220-135-151-1.hinet-ip.hinet.net
Jun	15	02:04:59	combo	sshd(pam_unix)[20893]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=220-135-151-1.hinet-ip.hinet.net user=root	root	220-135-151-1.hinet-ip.hinet.net
Jun	15	02:04:59	combo	sshd(pam_unix)[20896]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=220-135-151-1.hinet-ip.hinet.net user=root	root	220-135-151-1.hinet-ip.hinet.net
Jun	15	02:04:59	combo	sshd(pam_unix)[20897]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=220-135-151-1.hinet-ip.hinet.net user=root	root	220-135-151-1.hinet-ip.hinet.net
Jun	15	02:04:59	combo	sshd(pam_unix)[20898]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=220-135-151-1.hinet-ip.hinet.net user=root	root	220-135-151-1.hinet-ip.hinet.net
Jun	15	04:06:18	combo	su(pam_unix)[21416]:	session opened for user cyrus by (uid=0)	cyrus	
Jun	15	04:06:19	combo	su(pam_unix)[21416]:	session closed for user cyrus	cyrus	
Jun	15	04:06:20	combo	logrotate:	ALERT exited abnormally with [1]		
Jun	15	04:12:42	combo	su(pam_unix)[22644]:	session opened for user news by (uid=0)	news	
Jun	15	04:12:43	combo	su(pam_unix)[22644]:	session closed for user news	news	
Jun	15	12:12:34	combo	sshd(pam_unix)[23397]:	check pass; user unknown	unknown	
Jun	15	12:12:34	combo	sshd(pam_unix)[23397]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=218.188.2.4	unknown	218.188.2.4
Jun	15	12:12:34	combo	sshd(pam_unix)[23395]:	check pass; user unknown	unknown	
Jun	15	12:12:34	combo	sshd(pam_unix)[23395]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=218.188.2.4	unknown	218.188.2.4
Jun	15	12:12:34	combo	sshd(pam_unix)[23404]:	check pass; user unknown	unknown	
Jun	15	12:12:34	combo	sshd(pam_unix)[23404]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=218.188.2.4	unknown	218.188.2.4
Jun	15	12:12:34	combo	sshd(pam_unix)[23399]:	check pass; user unknown	unknown	
Jun	15	12:12:34	combo	sshd(pam_unix)[23399]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=218.188.2.4	unknown	218.188.2.4
Jun	15	12:12:34	combo	sshd(pam_unix)[23406]:	check pass; user unknown	unknown	
Jun	15	12:12:34	combo	sshd(pam_unix)[23406]:	authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=218.188.2.4	unknown	218.188.2.4
Jun	15	12:12:34	combo	sshd(pam_unix)[23396]:	check pass; user unknown	unknown	

Linux_2k.xlsx — 2,000 log lines organised into eight filterable columns.

Step 5: Summarise the Findings

The manual review of the opening section of the log produced the following assessment:

The logs show repeated failed login attempts from 218.188.2.4 and 220-135-151-1.hinet-ip.hinet.net, concentrated almost entirely on the root account. The repetition and the tight timing point to automated brute-force activity rather than a user mistyping a password. The accompanying "check pass; user unknown" messages indicate the attacker is also guessing usernames that do not exist on the host, which is characteristic of scripted enumeration. Separately, the entry "logrotate: ALERT exited abnormally with [1]" is unrelated to the authentication activity but warrants its own investigation, as a failing log-rotation service can silently interrupt the very logging this analysis depends on.

Skills Learned

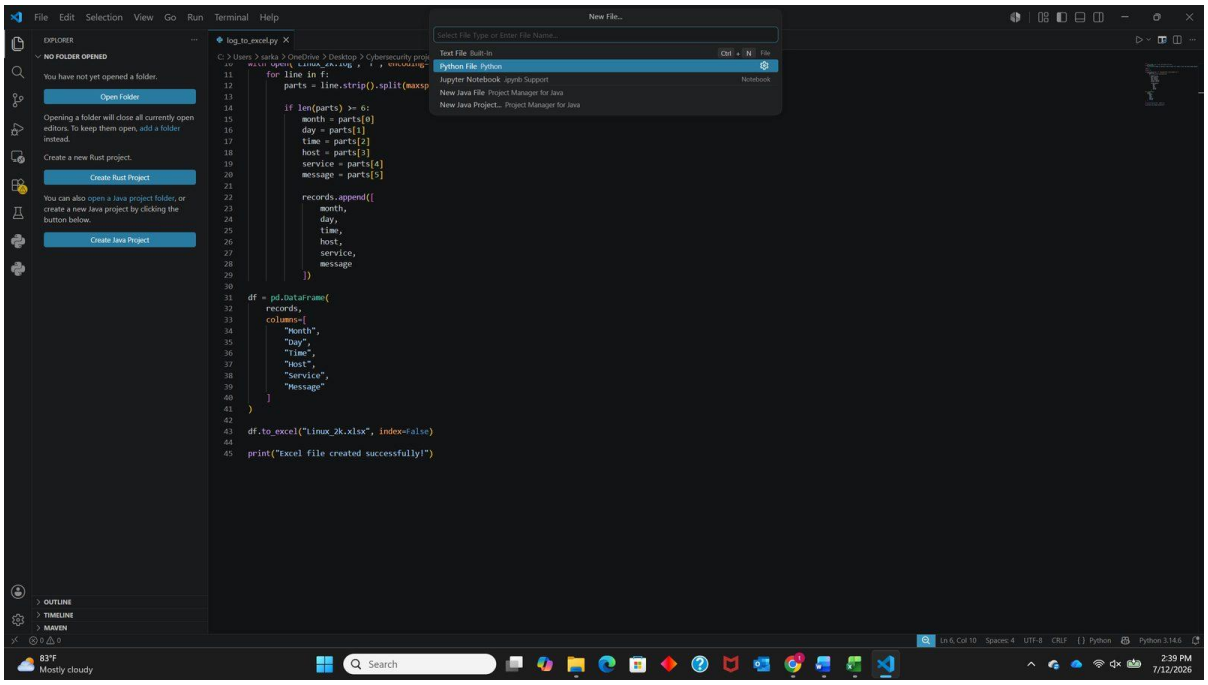
- Working with a real-world Linux log dataset rather than synthetic samples
- Identifying suspicious events: repeated failed logins, unknown users, and system alerts
- Recognising the syslog message structure that makes automated field extraction possible
- Organising and tabulating findings so that patterns become countable
- Summarising insights in the style of a SOC analyst report

Objective 2 — Automating Log Analysis

Manual review works for 40 lines and becomes unreliable well before 2,000. This objective replaces the eye with a script: a Python program that reads the same log file, classifies every line against a set of detection patterns, reports what it finds, and writes the results to CSV for further analysis. This is the point at which the workflow becomes repeatable — the same script run against a new log file produces the same analysis in seconds.

Step 1: Create the Python File

A project folder was created in Visual Studio Code containing Log_Analysis_Automation.py alongside the log file itself. Keeping the script and Linux_2k.log in the same directory means the script can reference the file by name without any path configuration, which removes a common source of errors.



Creating the analysis script in Visual Studio Code.

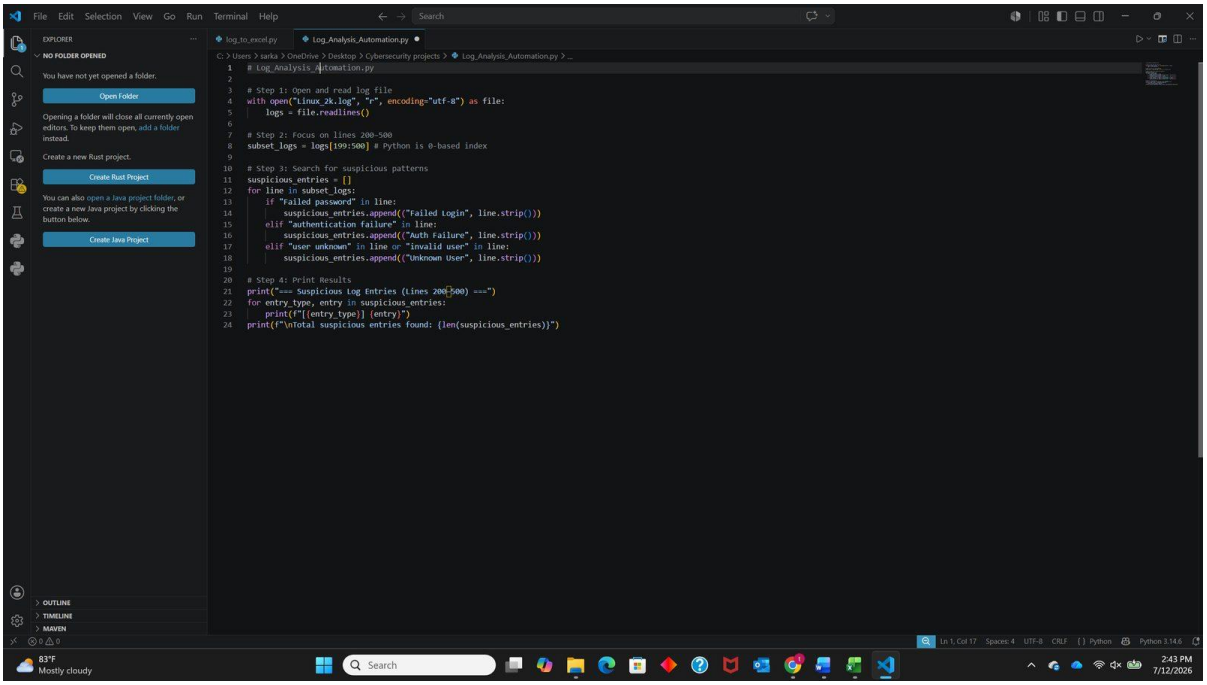
Step 2: Write the Log Analysis Script

The script reads the log into memory, iterates over the selected range of lines, tests each line against three detection patterns, and tags any match with an event type. The classification is deliberately simple and readable — it mirrors exactly the three categories identified manually in Objective 1.

```
# Log_Analysis_Automation.py

# Step 1: Open and read log file
with open("Linux_2k.log", "r", encoding="utf-8") as file:
    logs = file.readlines()

# Step 2: Select the range of lines to analyse
subset_logs = logs[0:5000]          # full file; use logs[199:500] for lines 200-500

# Step 3: Search for suspicious patterns
suspicious_entries = []
for line in subset_logs:
```

```
    if "Failed password" in line:
        suspicious_entries.append(("Failed Login", line.strip()))
    elif "authentication failure" in line:
        suspicious_entries.append(("Auth Failure", line.strip()))
    elif "user unknown" in line or "invalid user" in line:
        suspicious_entries.append(("Unknown User", line.strip()))

# Step 4: Print results
print("=== Suspicious Log Entries ===")
for entry_type, entry in suspicious_entries:
    print(f"[{entry_type}] {entry}")
print(f"\nTotal suspicious entries found: {len(suspicious_entries)}")
```

The script opens Linux_2k.log and reads its lines into a list. It then checks each line for the keywords “Failed password”, “authentication failure”, and “user unknown” or “invalid user”. Any matching line is appended to suspicious_entries together with a label describing the event type, and the script prints each flagged entry followed by a total count. What took several minutes of careful reading in Objective 1 now completes instantly across the entire file.



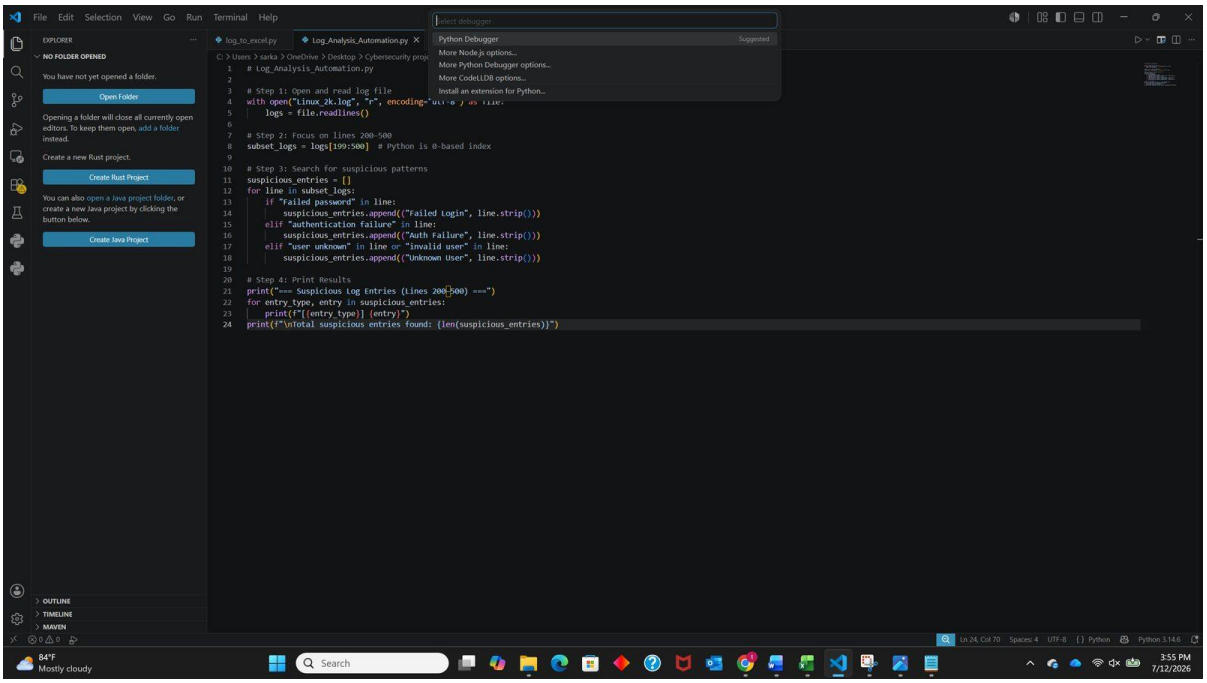
Log_Analysis_Automation.py — the detection logic.

Step 3: Run the Script

With Linux_2k.log in the same folder, the script is executed from the Visual Studio Code terminal:

```
python Log_Analysis_Automation.py
```

The output lists every flagged entry with its classification, then reports the total. Run against the full file, the script identified 607 suspicious entries out of 2,000 lines — 30.4% of the log. Run against the narrower range of lines 200 to 500 used during initial testing, it identified 140.



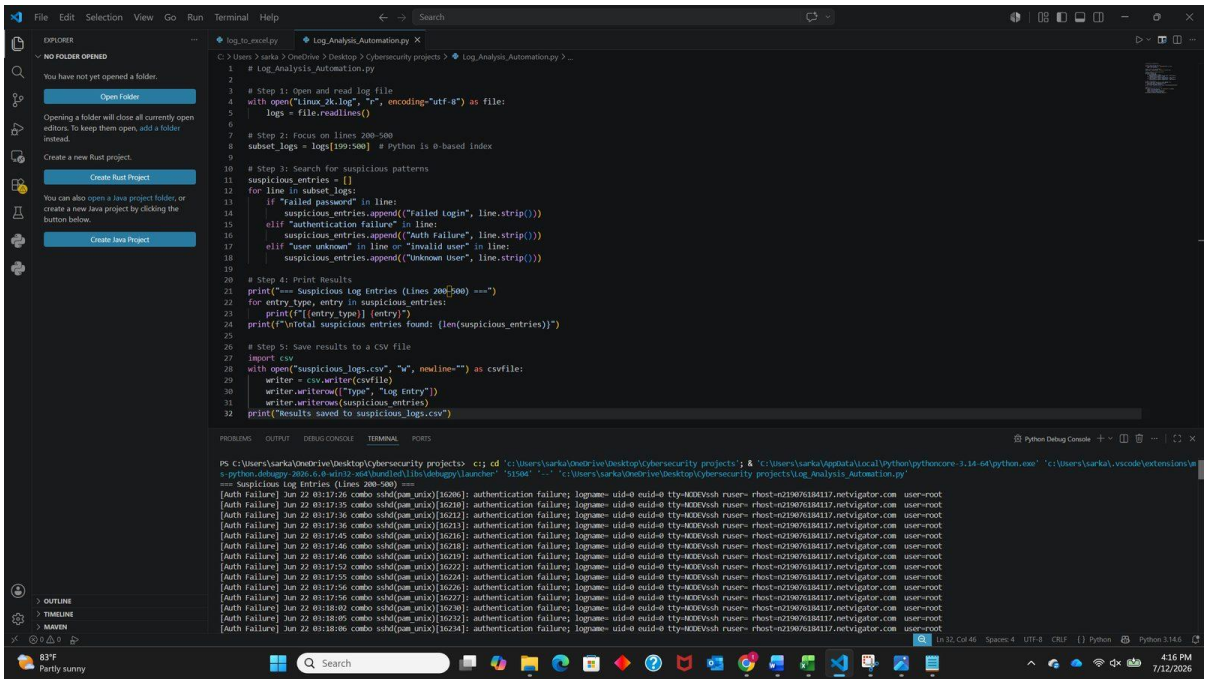
Script output — flagged entries with their event classification.

Step 4: Export the Results

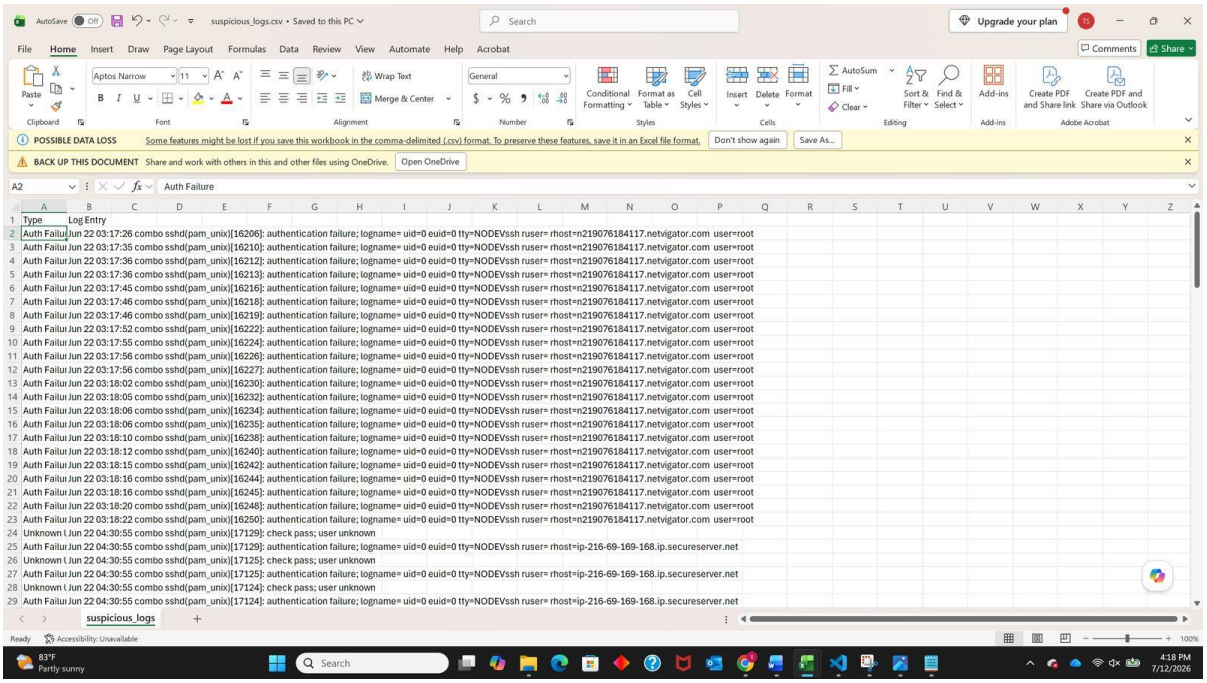
Printing to a terminal is useful for a quick check but nothing downstream can consume it. Appending a CSV export turns the output into a structured artefact that opens directly in Excel or Google Sheets, and that can be attached to a ticket or handed to another analyst.

```
# Step 5: Save results to a CSV file
import csv

with open("Full_suspicious_logs.csv", "w", newline="") as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow(["Type", "Log Entry"])
    writer.writerows(suspicious_entries)

print("Results saved to Full_suspicious_logs.csv")
```

Two output files were produced: Full_suspicious_logs.csv containing all 607 entries from the complete file, and suspicious_logs.csv containing the 140 entries from the lines 200–500 range. Both are included in the repository.



The CSV export step appended to the script.



Full_suspicious_logs.csv opened in Excel — 607 classified entries.

Skills Learned

- Writing Python for a security task rather than a general programming exercise
- Selecting and slicing specific log ranges for targeted analysis
- Automating detection of authentication failures and unknown-user probes

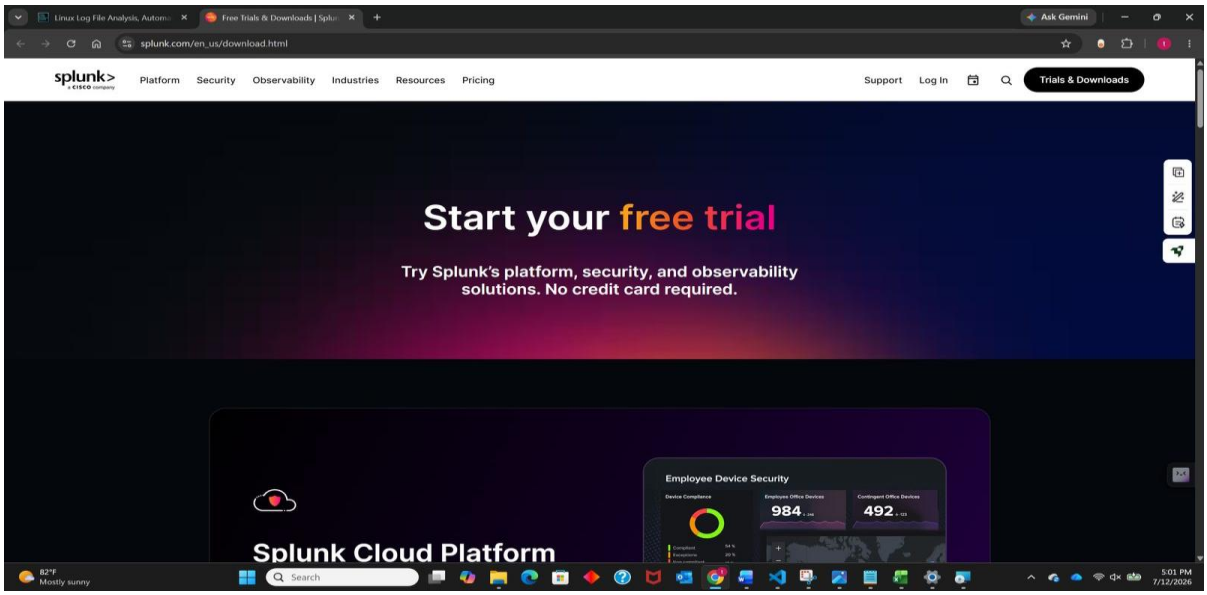
- Generating a CSV report that feeds directly into spreadsheet or SIEM workflows
- Understanding the trade-off between simple keyword matching and format-aware parsing

Objective 3 — Log Analysis and Visualization with Splunk

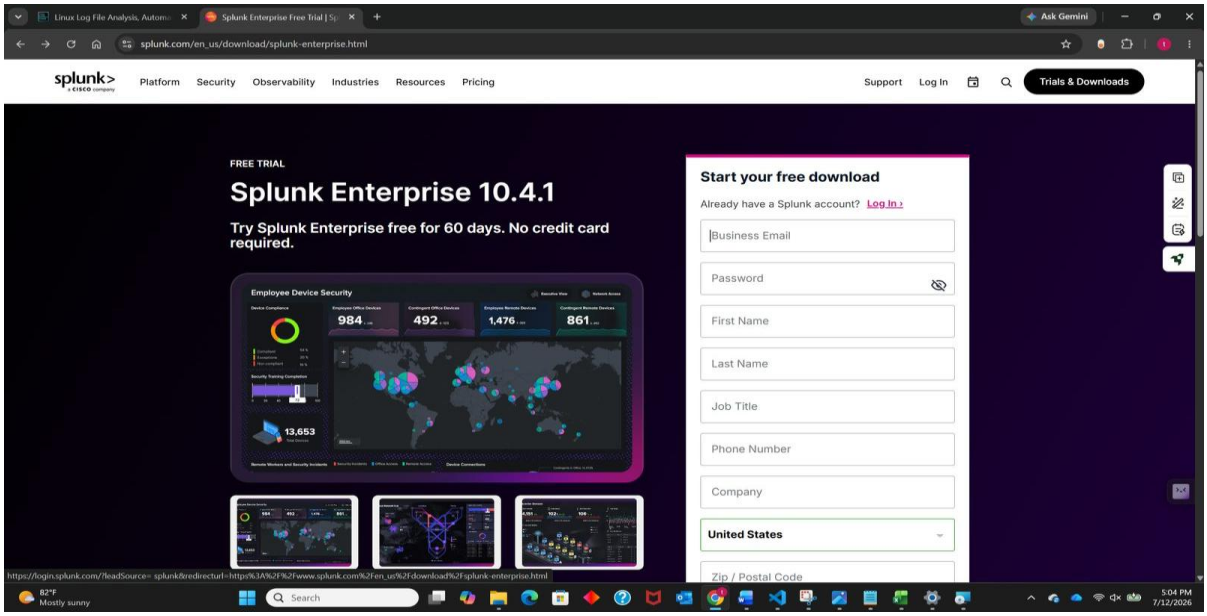
A Python script answers questions it was written to answer. A SIEM answers questions asked after the data is already loaded. This objective ingests the same Linux_2k.log into Splunk Enterprise — the platform used in professional SOC environments — and uses it to query, correlate, and visualise the activity at a scale that neither of the previous two approaches can reach.

Step 1: Download and Set Up Splunk

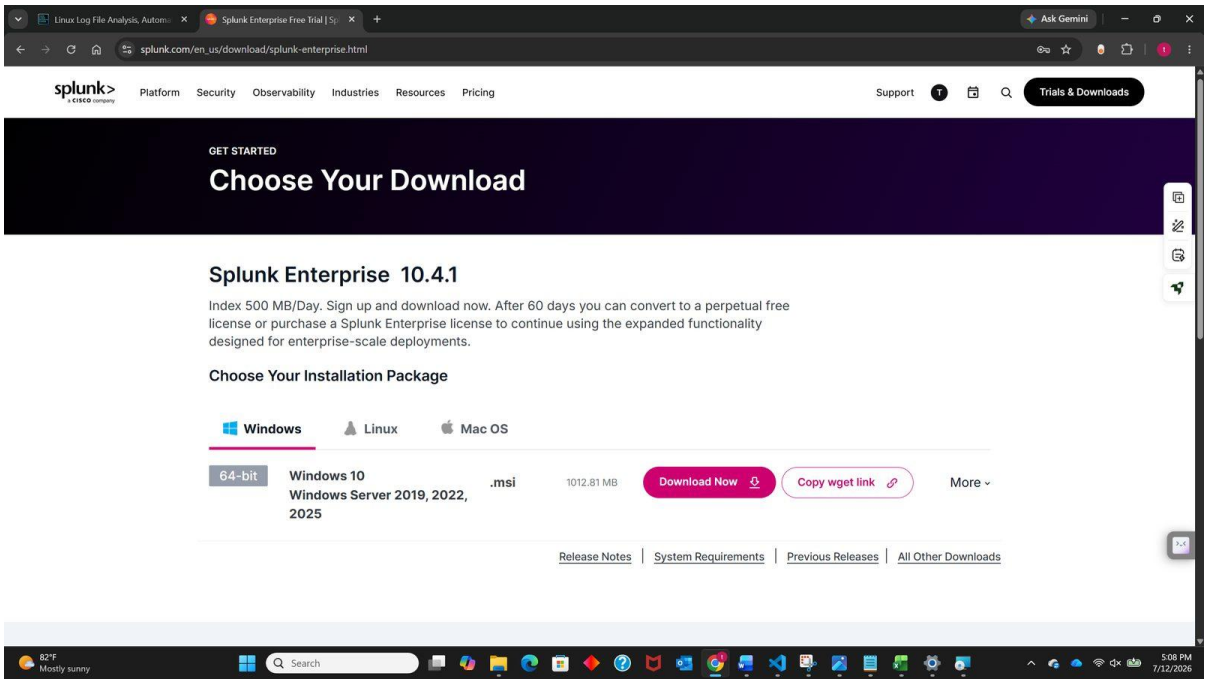
Splunk Enterprise is available as a free 60-day trial from splunk.com. Registration is required to download; the Windows installation package was selected for this lab. During installation an administrator account is created — these credentials are used to reach the Splunk web interface at 127.0.0.1:8000.



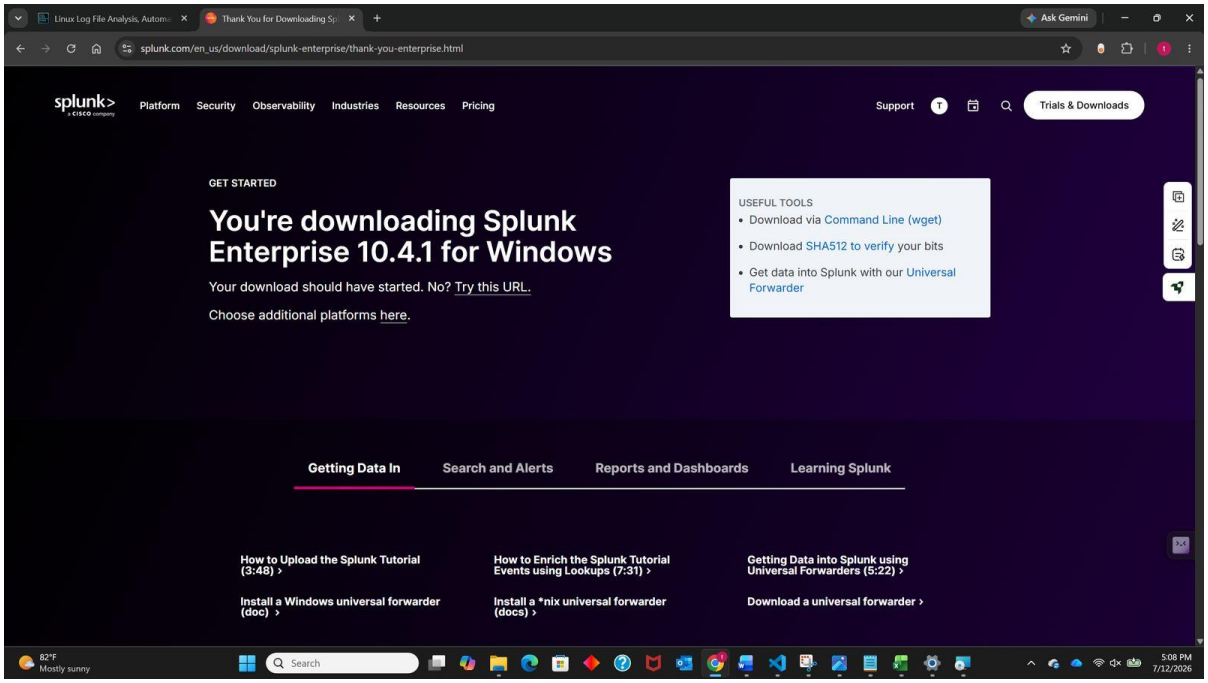
The Splunk Enterprise free trial page.



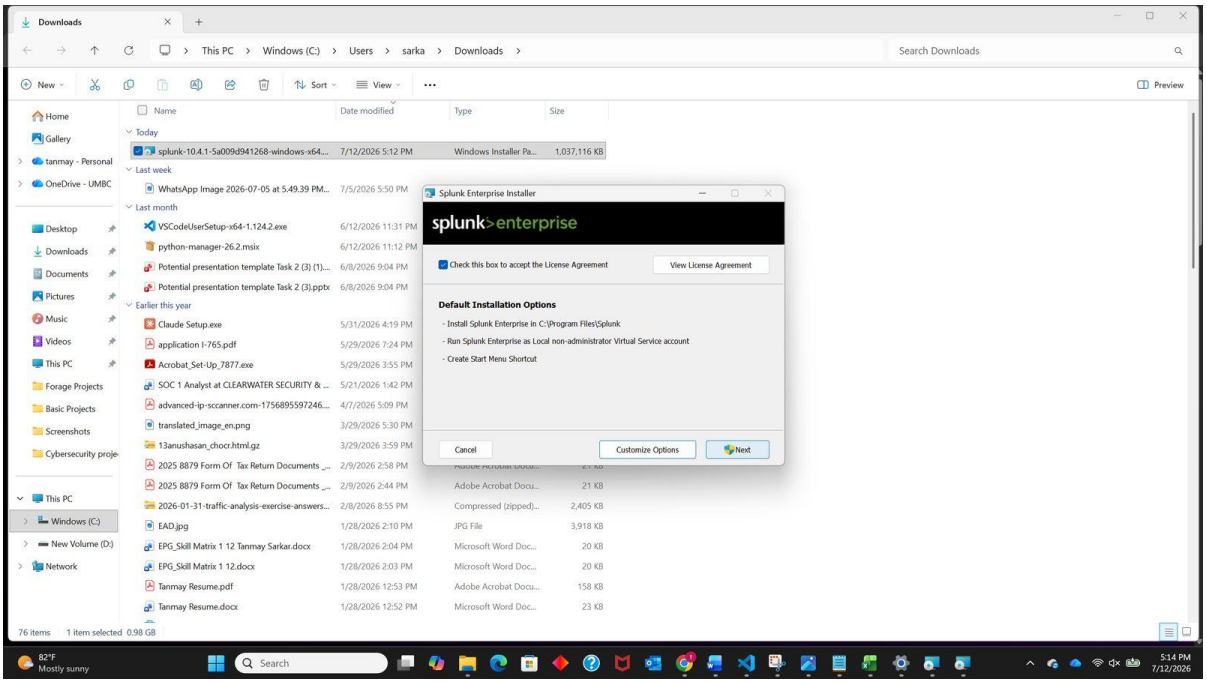
Splunk Enterprise 10.4.1 download page.



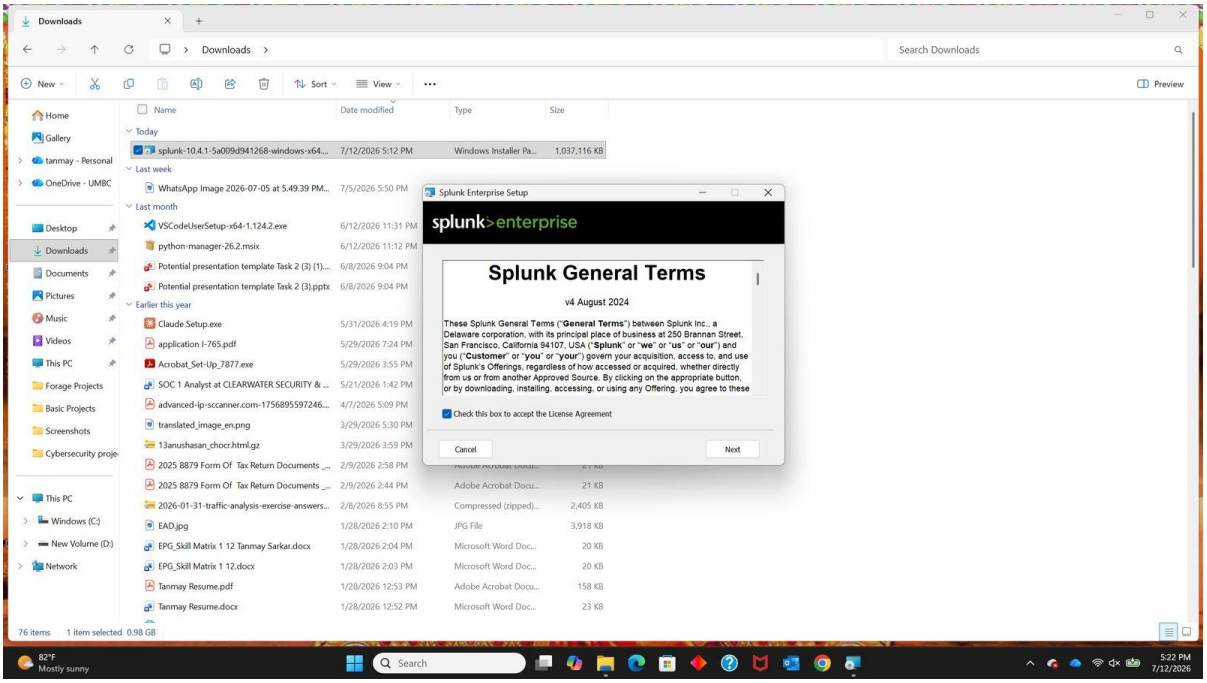
Selecting the Windows installation package.



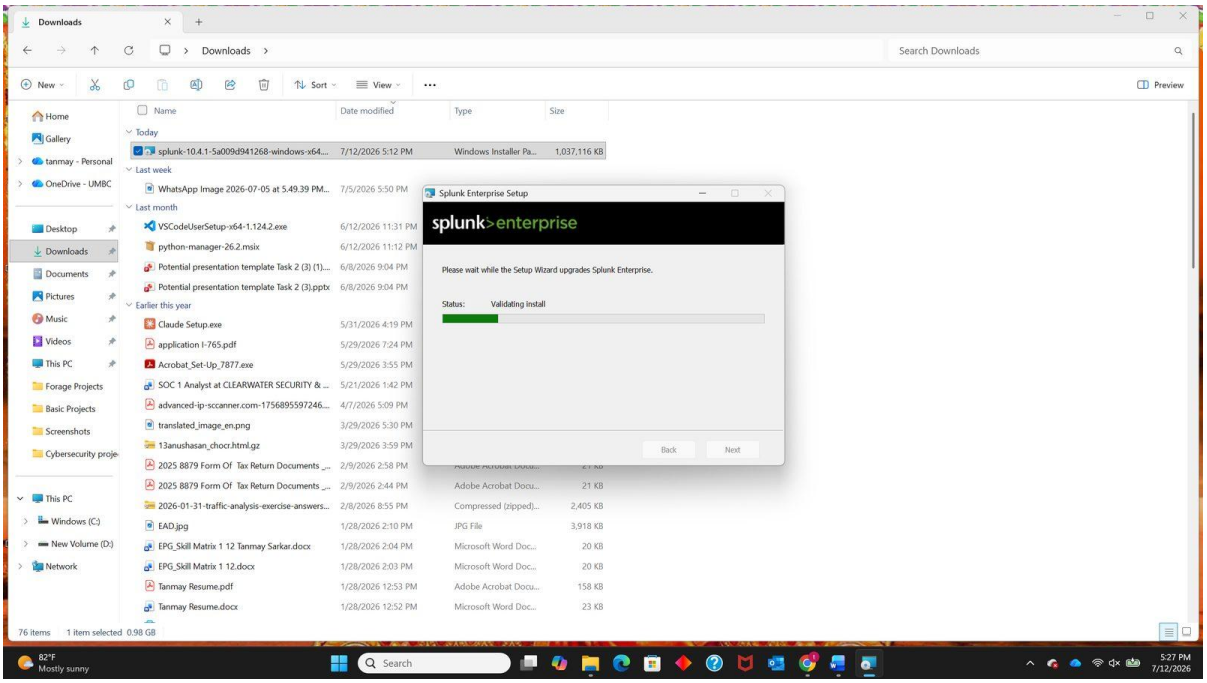
Download starting.



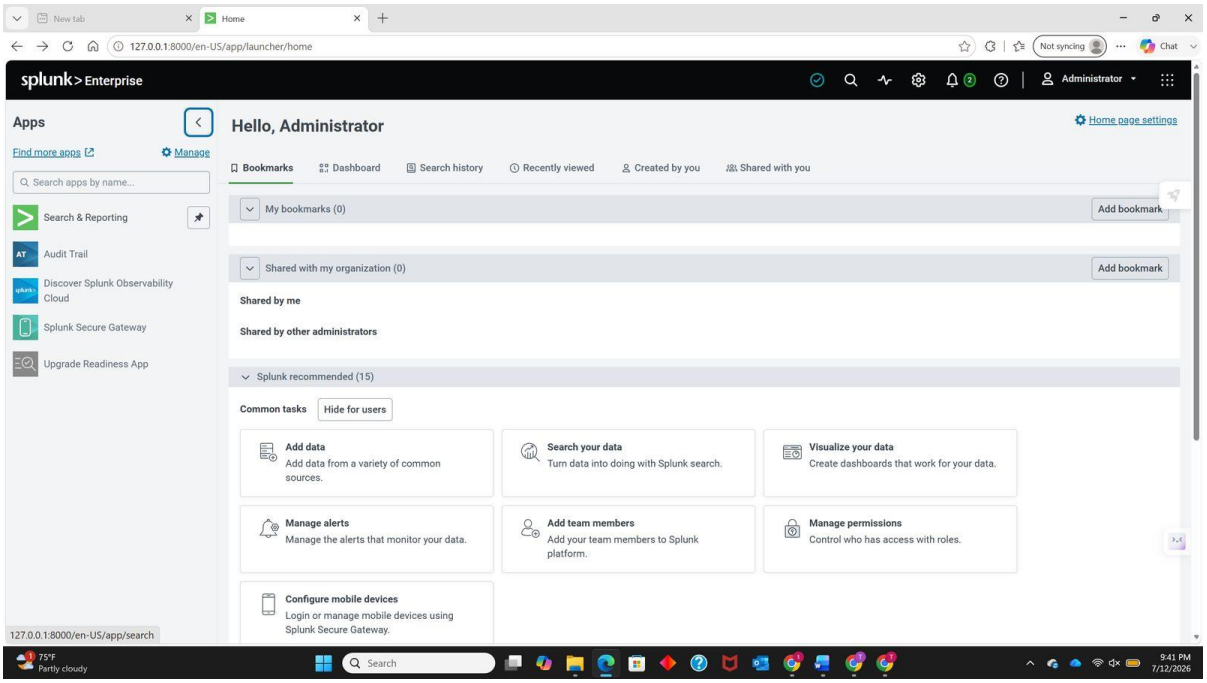
The downloaded installer.



Accepting the Splunk General Terms during installation.



Installation in progress.



The Splunk Enterprise administrator dashboard after first sign-in.

Step 2: Upload the Log File

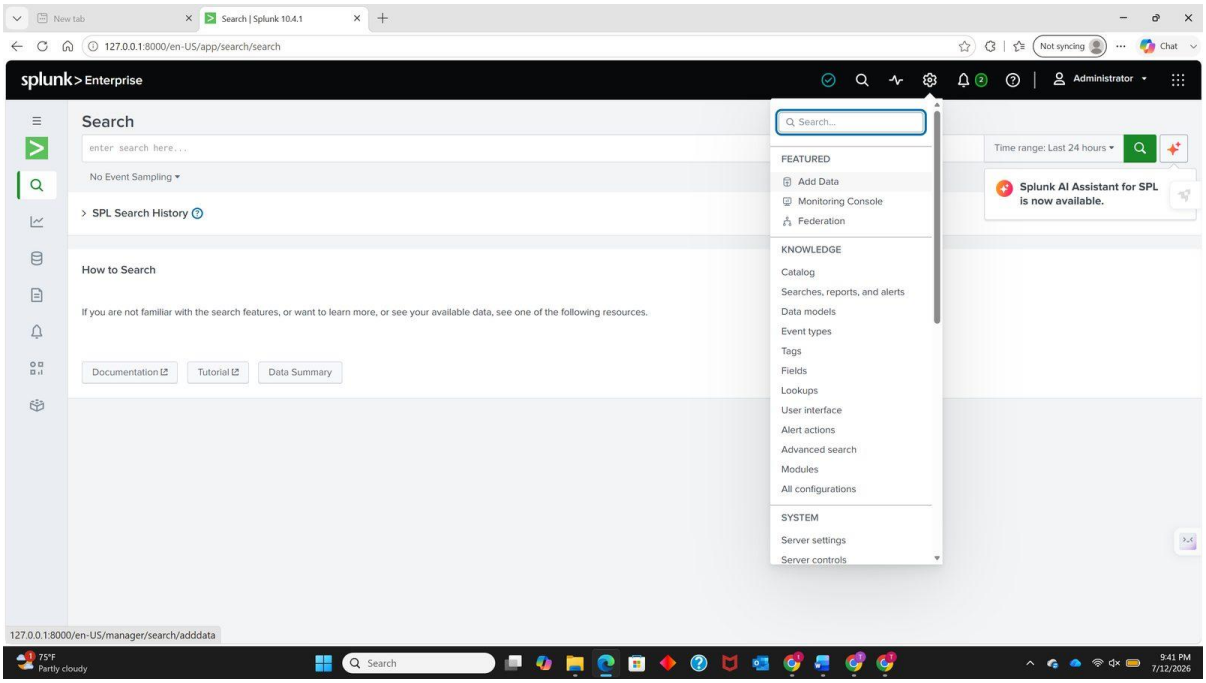
Data is brought into Splunk through Settings → Add Data → Upload. The same Linux_2k.log used in Objectives 1 and 2 was selected so that all three stages analyse identical data and the results are directly comparable.

Splunk auto-detected the format and the source type was saved with the following configuration:

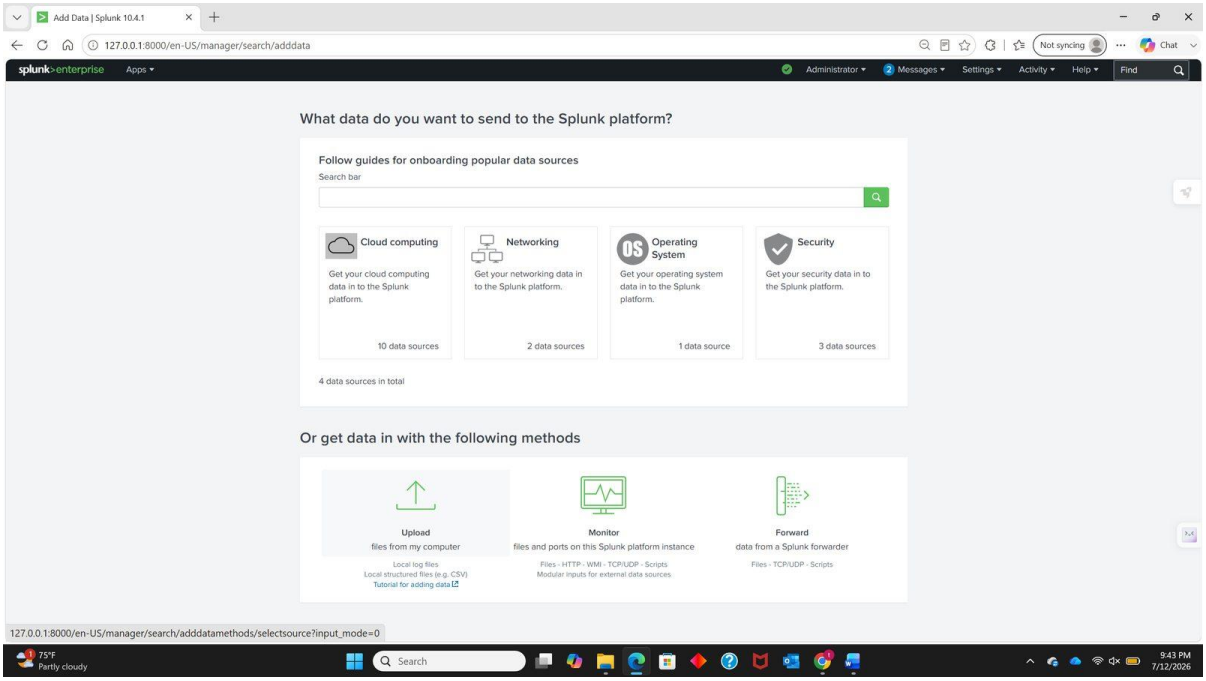
- Source Type: default auto-detected
- Name: Linux
- Description: Log Analysis
- Category: Custom
- App: Search & Reporting

In Input Settings, the Host was set to a constant value (tsarkar1 — this will differ depending on the machine used) and the Default index was selected. These settings ensure the events are correctly labelled and searchable after ingestion.

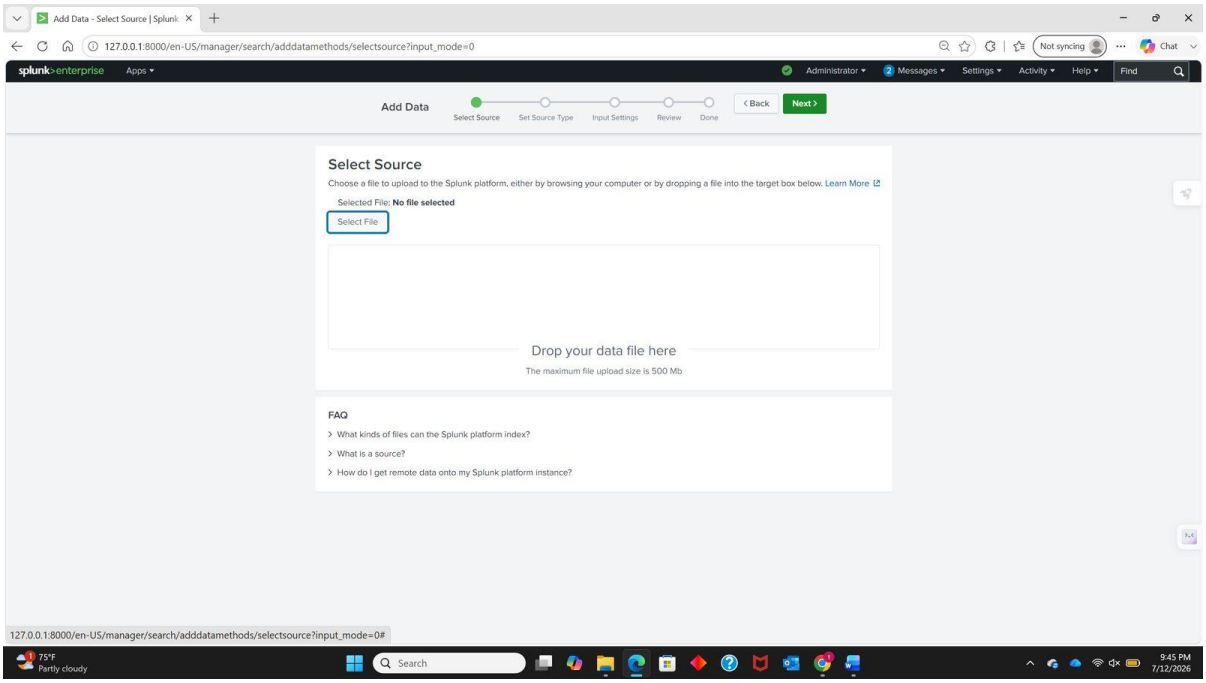
Troubleshooting note: if the upload fails with “Upload failed with ERROR: can only concatenate str (not 'NoneType') to str”, rename the file so it contains no spaces or special characters and repeat the process. Splunk’s upload handler can fail on non-standard characters in the path or filename.



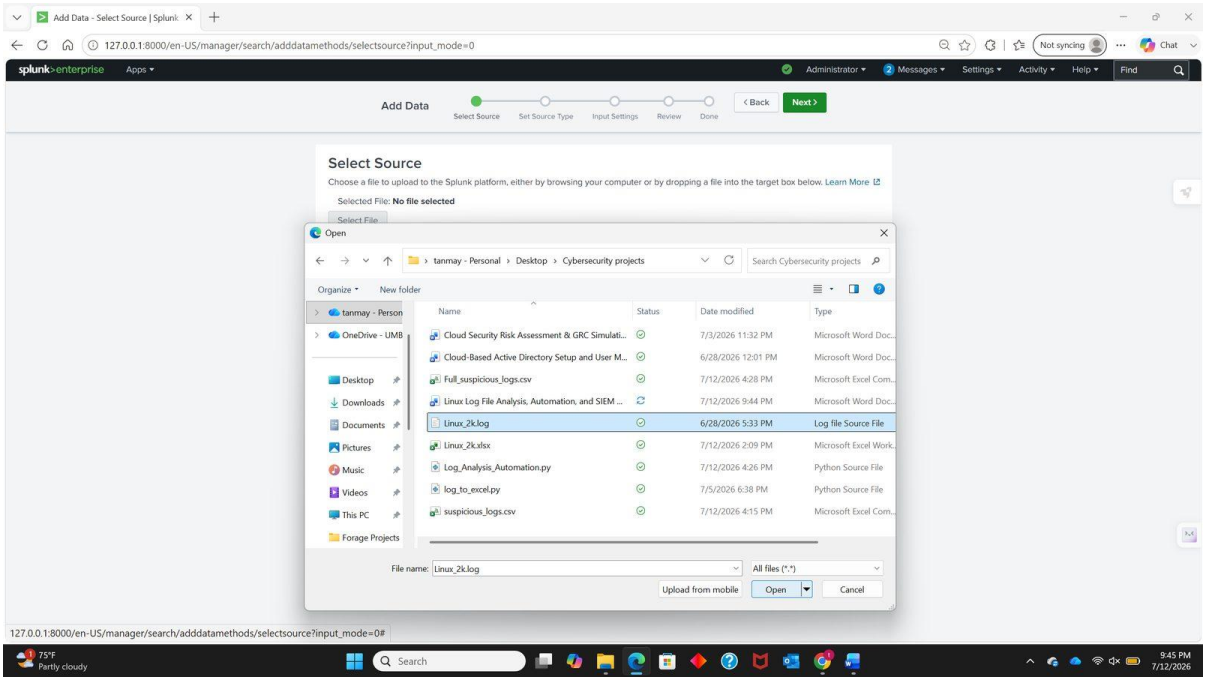
Settings → Add Data.



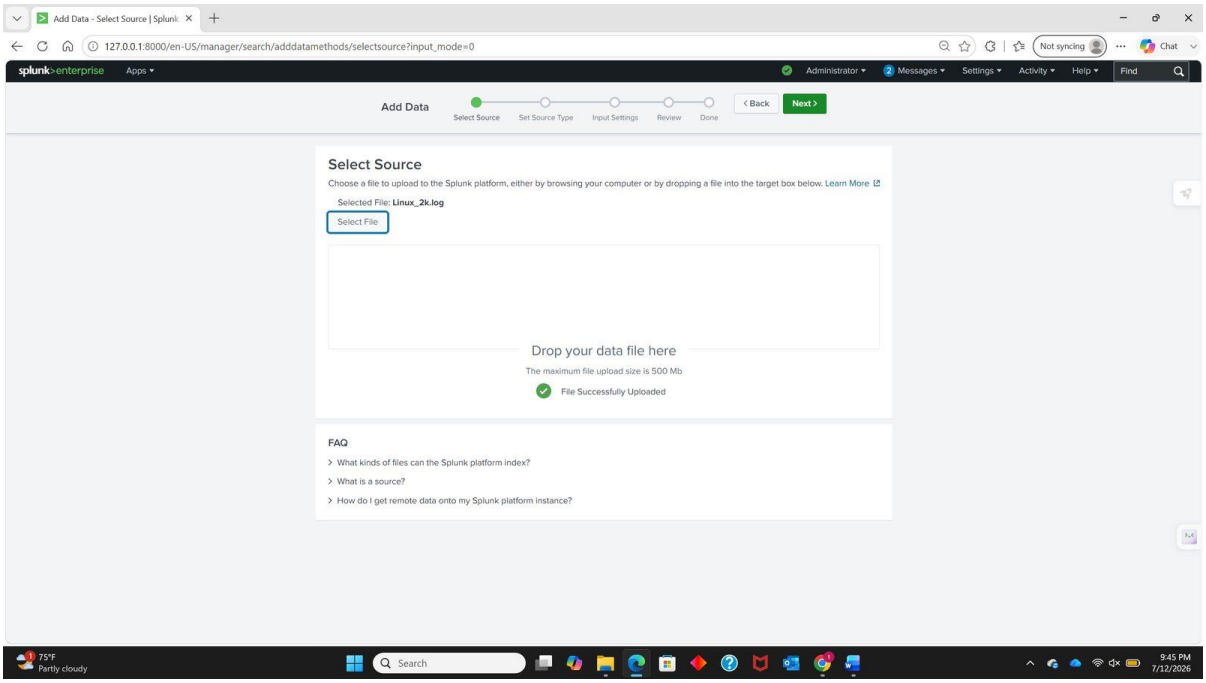
The Add Data workflow.



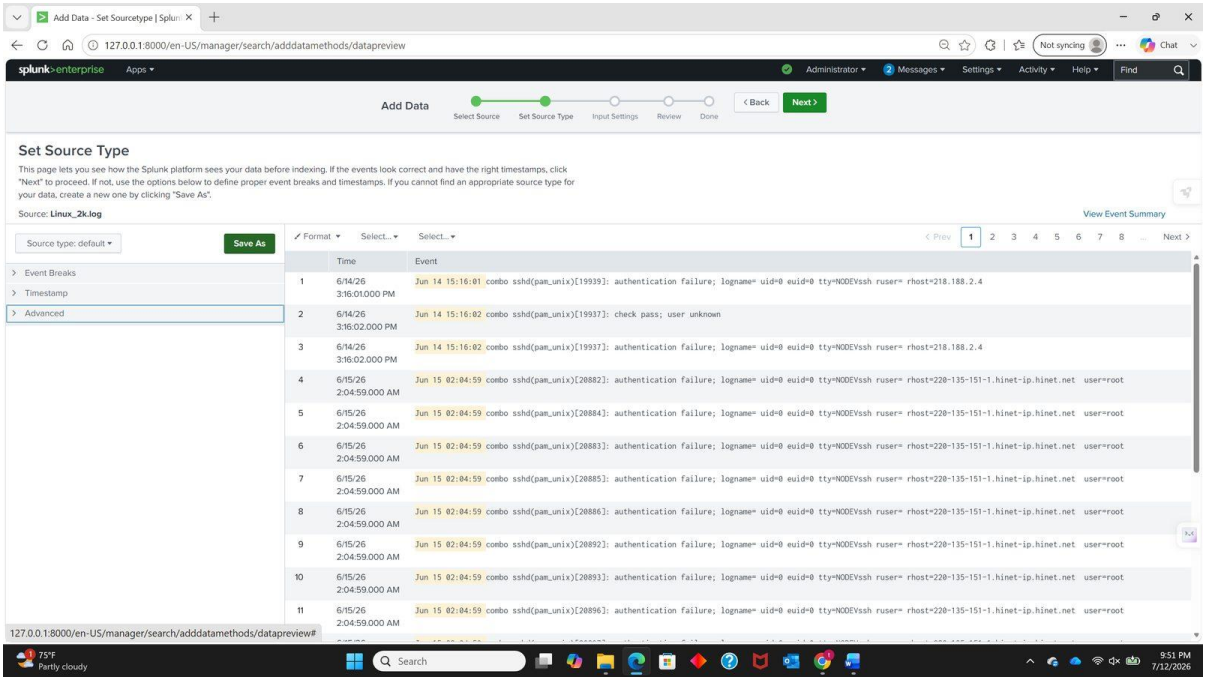
Selecting Upload as the data source.



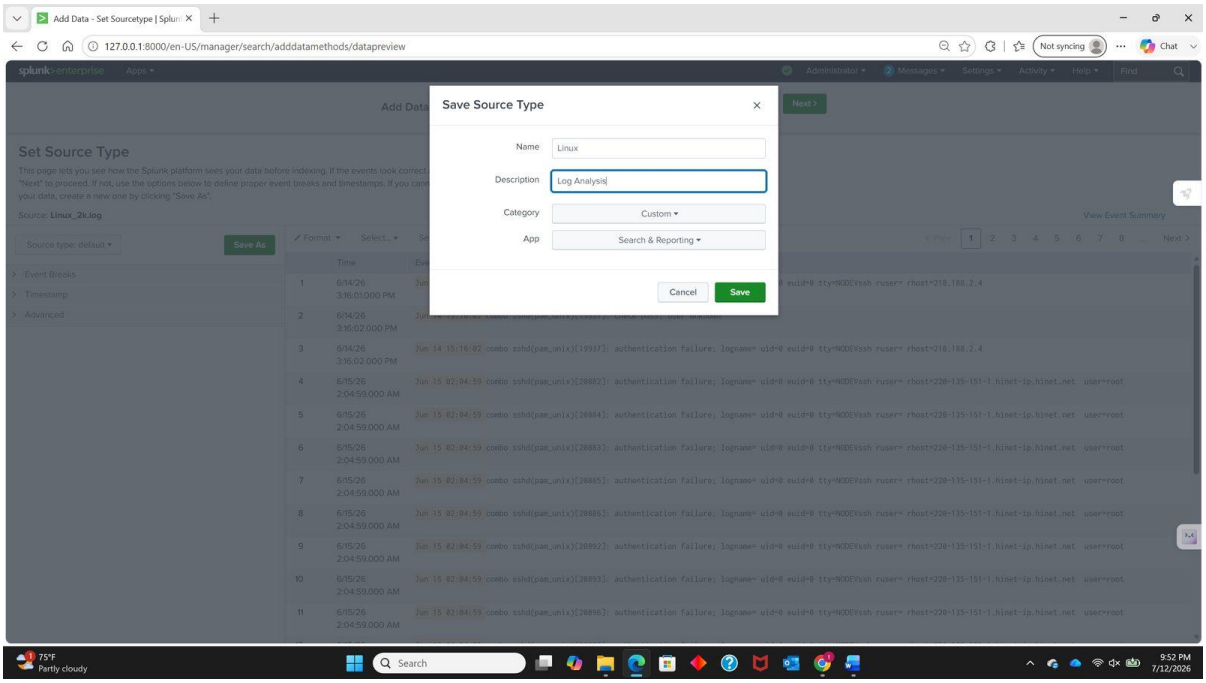
Choosing Linux_2k.log from disk.



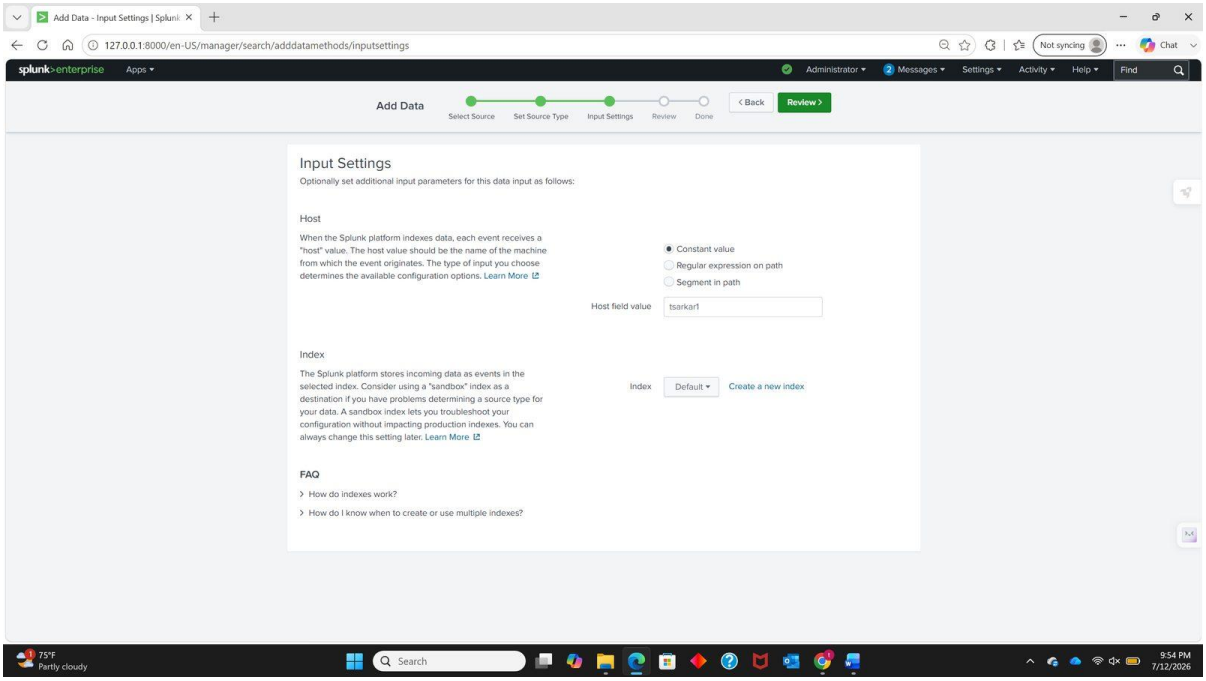
The file selected and ready for source-type detection.



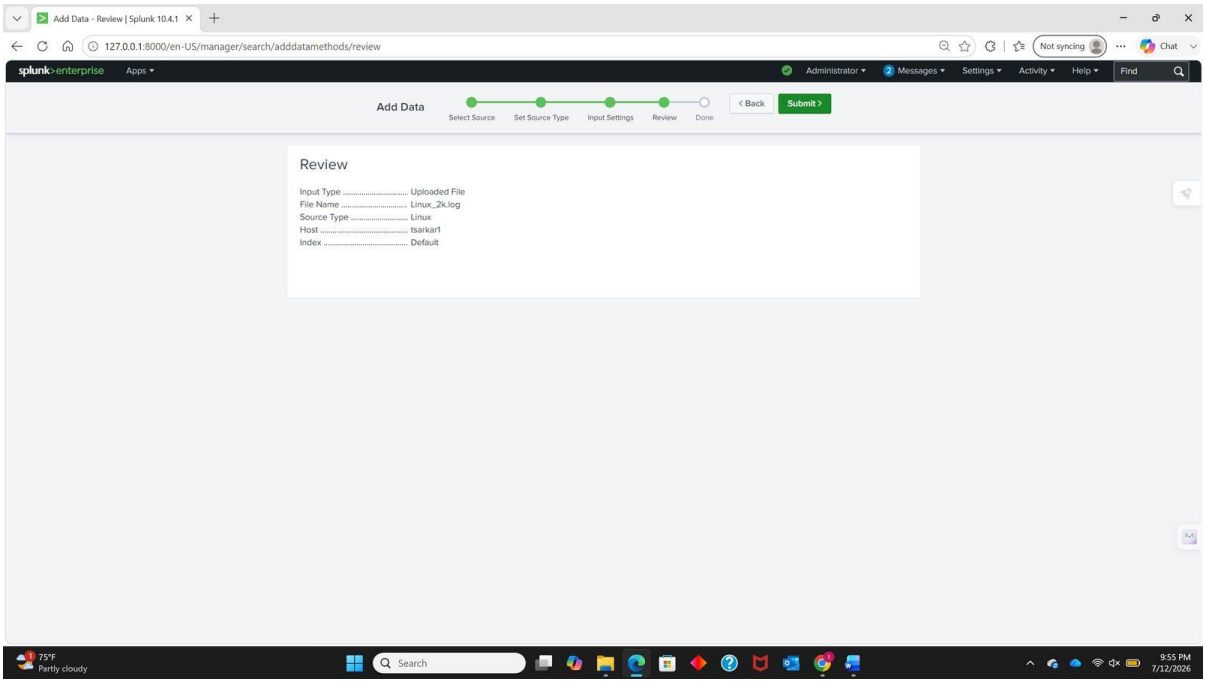
Splunk auto-detecting the source type.



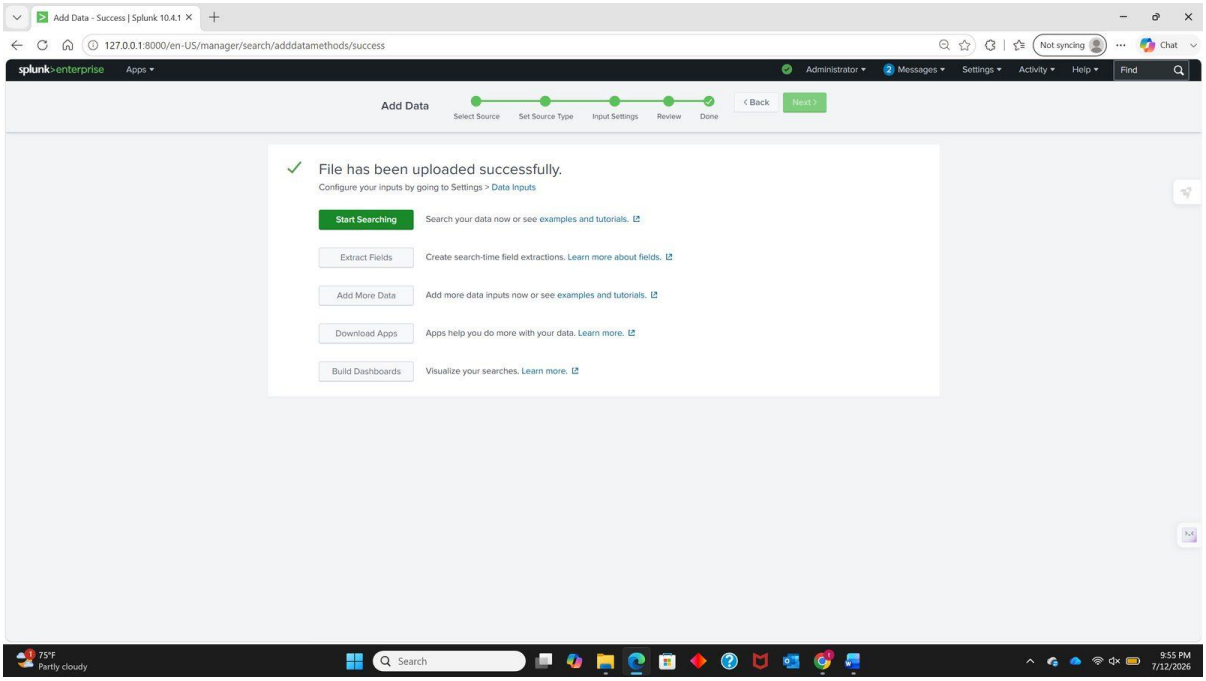
Saving the source type as "Linux".



Input Settings — host set to a constant value, default index selected.



Reviewing the configuration before submission.



Upload complete — the file has been indexed successfully.

Step 3: Search and Filter Suspicious Activity

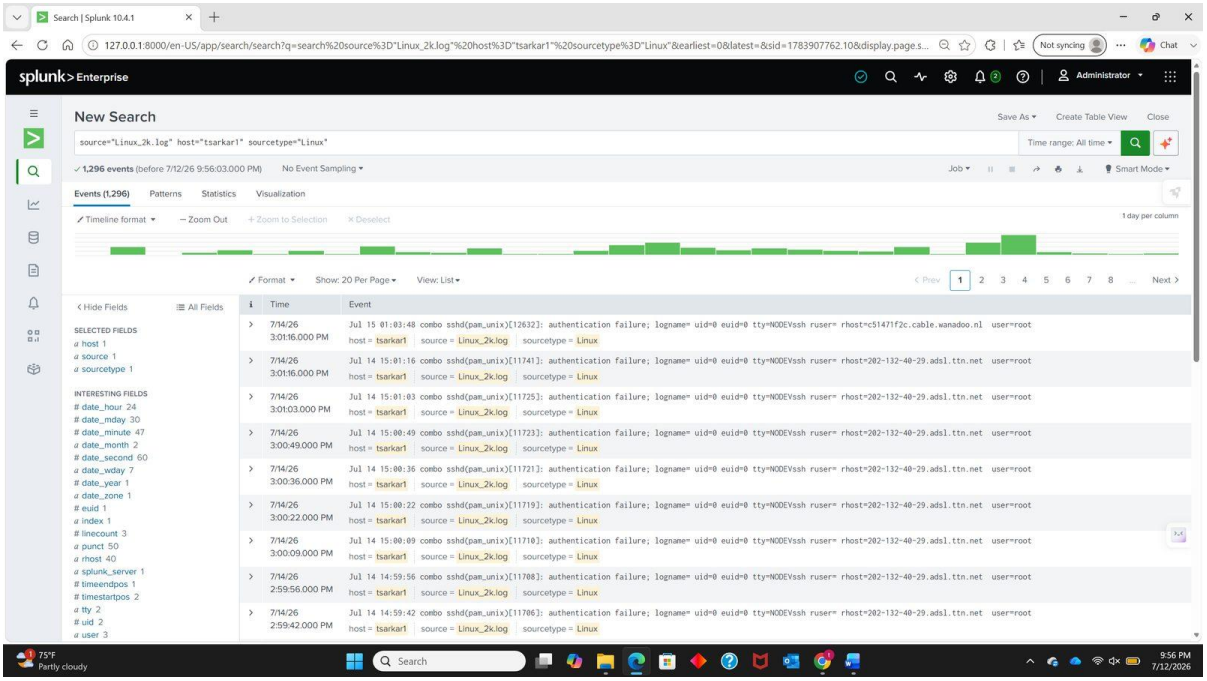
Once indexing finished, the default search metadata for this dataset was:

```
source="Linux_2k.log" host="tsarkar1" sourcetype="Linux"
```

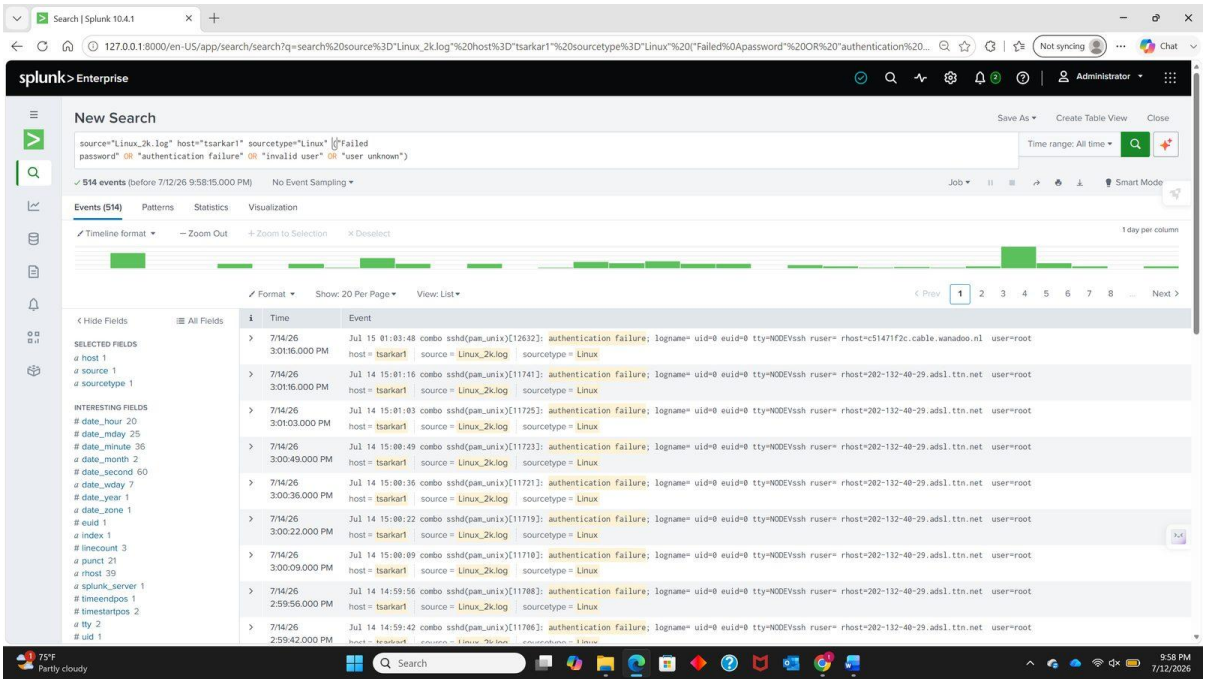
This returns all 1,296 indexed events. To isolate authentication activity, the same three detection patterns used in the Python script were applied as a search filter — plus “Failed password”, which the script also tested for:

```
source="Linux_2k.log" host="tsarkar1" sourcetype="Linux"
("Failed password" OR "authentication failure" OR "invalid user" OR "user unknown")
```

The filter narrowed 1,296 events to 514 — every event matching a known indicator of brute-force or unauthorised access. From here the timestamps, usernames, and source addresses can be reviewed for pattern.



The base search — 1,296 events indexed.



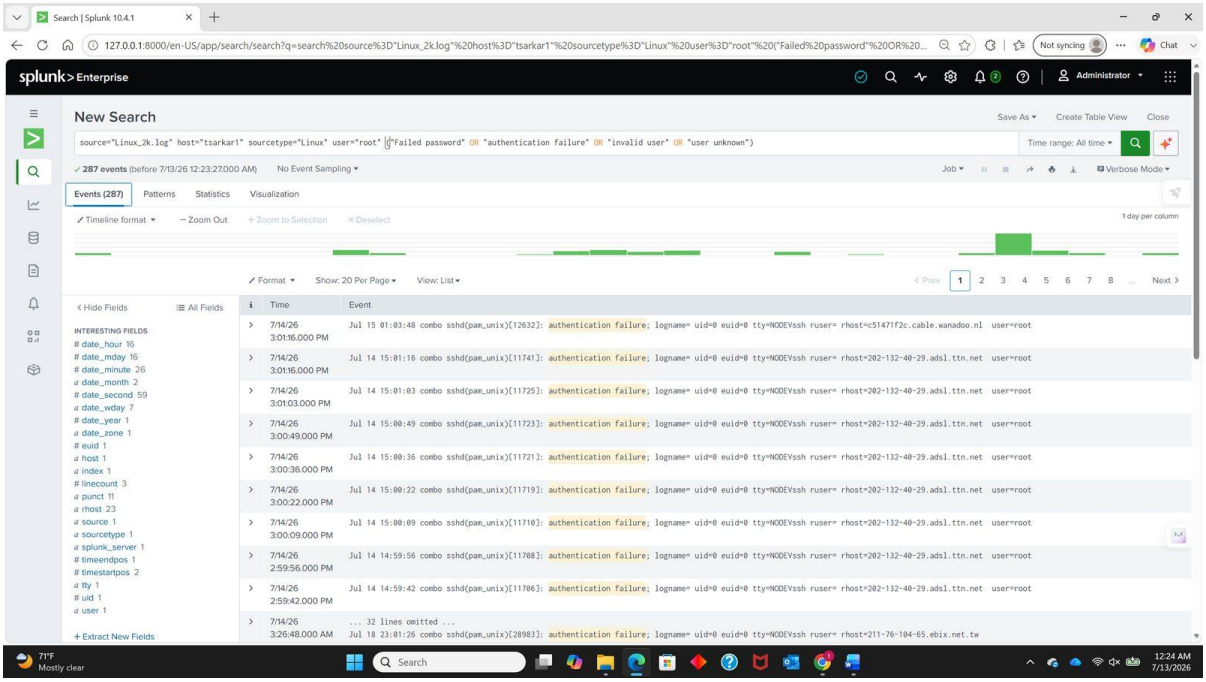
The filtered search — 514 events matching authentication-failure indicators.

Step 4: Visualise the Data

Splunk presents the same result set through four views, each answering a different analytical question. Working through them in order is the practical shape of an investigation: find it, confirm it repeats, measure it, then show it.

Events View — identifying the suspicious source

The Events view lists individual records. Scrolling through the filtered results, the same pattern appears repeatedly: authentication failures against user=root, arriving from a single rhost, seconds apart. Root is a high-value target because it holds full administrative control, and repeated failures against it are never routine. At this point a SOC analyst would escalate for pattern analysis.



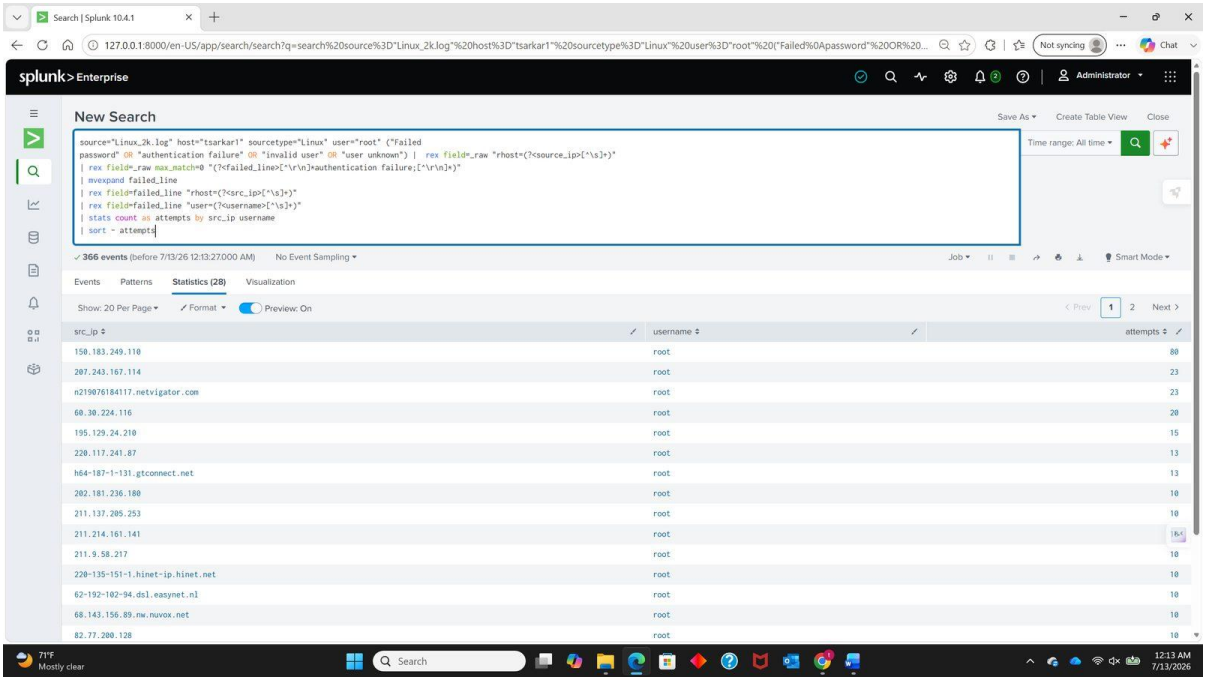
Events view — repeated authentication failures against root, highlighted.

Patterns View — confirming repetitive behaviour

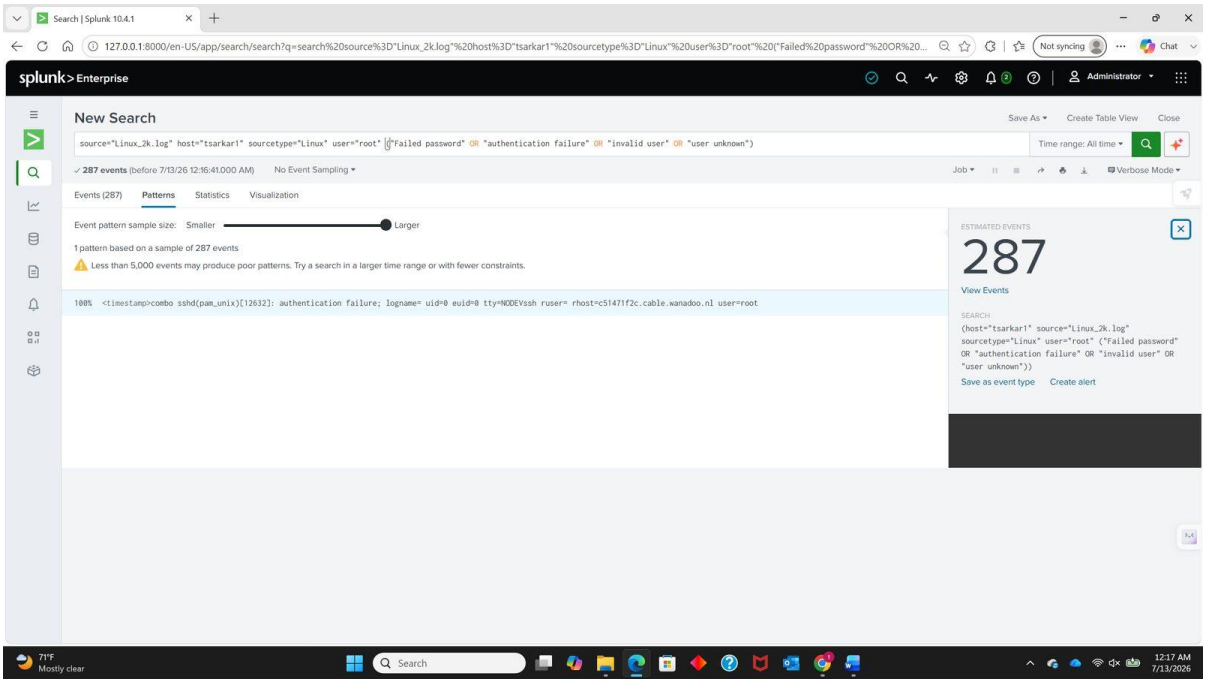
The Patterns view clusters similar events and shows what proportion of the result set each cluster represents. A single pattern — authentication failure against root over SSH — dominates the sample. This confirms the activity is systematic rather than incidental.

To make the event types countable rather than merely visible, field extraction was applied directly in SPL. This query extracts each failure string from the raw event, expands multi-value matches into individual rows, classifies them, and produces a sorted count:

```
source="Linux_2k.log" host="tsarkar1" sourcetype="Linux"
("Failed password" OR "authentication failure" OR "invalid user" OR "user unknown")
| rex field=_raw max_match=0 "(?<failed_line>[^\r\n]*(?:Failed password|authentication failure|invalid
user|user unknown)[^\r\n]*)"
| mvexpand failed_line
| eval authentication_type=case(
    like(failed_line,"%Failed password%"),"Failed password",
    like(failed_line,"%authentication failure%"),"Authentication failure",
    like(failed_line,"%invalid user%"),"Invalid user",
    like(failed_line,"%user unknown%"),"User unknown"
  )
| stats count as attempts by authentication_type
| sort - attempts
```



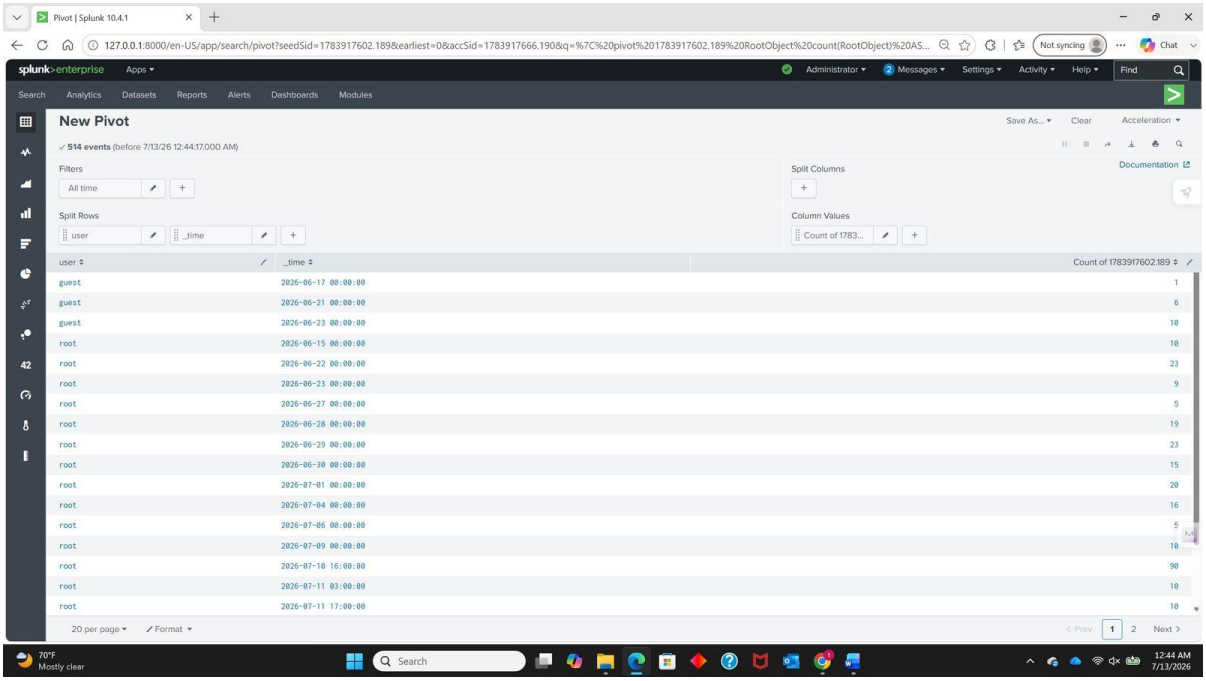
Field extraction and classification in SPL.



Patterns view — a single authentication-failure pattern dominates the sample.

Statistics View — breaking down the data

A Pivot was built over the filtered result set, splitting rows by user and by time and counting events in each bucket. The breakdown makes the concentration explicit: root appears in nearly every row, guest appears in a handful, and one time bucket stands far above the rest — 90 events against root within a single hour on 10 July. That single bucket is the largest attack burst in the dataset.



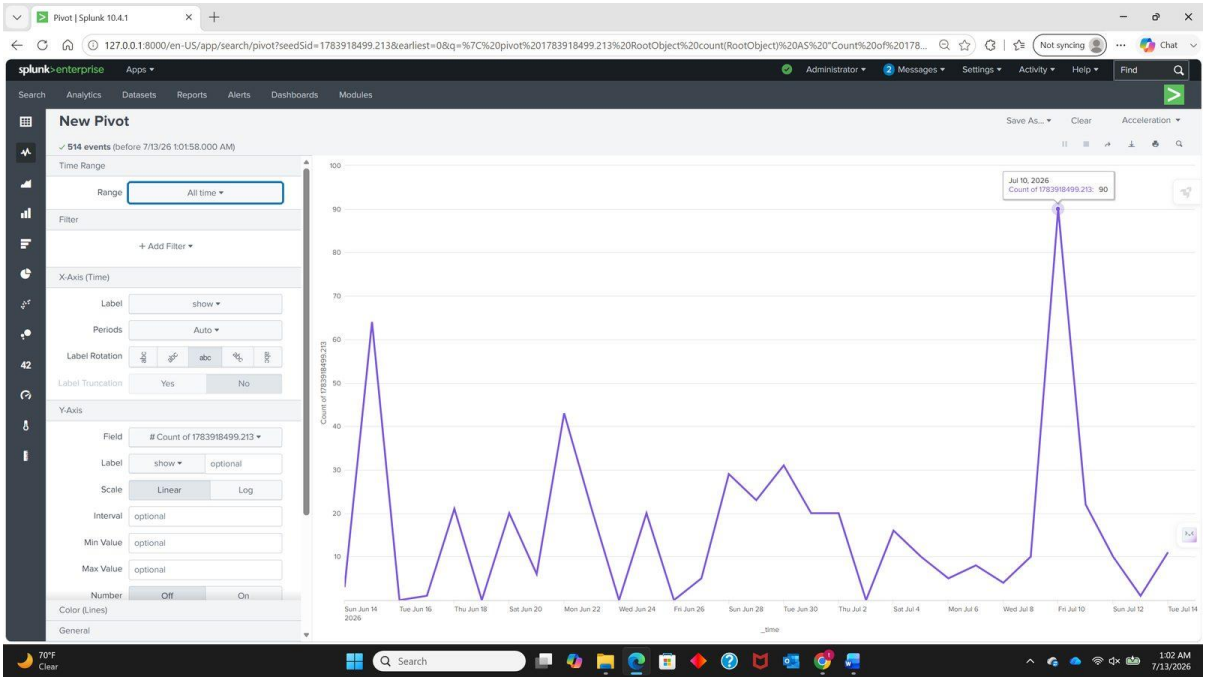
Pivot statistics — event counts by targeted user and time bucket.

Visualization View — presenting the attack timeline

Plotting the same counts as a line chart over time converts the table into a shape that can be read at a glance. The chart shows a low, irregular baseline of authentication failures across the whole period, punctuated by sharp spikes — most prominently the 10 July peak.

What this suggests:

- The activity is bursty rather than continuous, which is characteristic of automated tooling launched against a target, not of a user repeatedly mistyping a password
- The concentration on root indicates an attempt to obtain administrative access directly
- The pattern is consistent with scripted attacks or botnet traffic sweeping exposed SSH endpoints



Authentication failures over time — the 10 July spike reaches 90 events.

This is the value a SIEM adds over the previous two stages. The Python script could count 607 suspicious entries; it could not show that they arrive in concentrated bursts separated by weeks of quiet, and that shape is what distinguishes an automated campaign from background noise.

Skills Learned

- Ingesting and indexing log data in Splunk Enterprise, including source-type and input configuration
- Writing SPL searches to filter authentication-related events from a mixed log source
- Extracting fields from unstructured event text with rex, mvexpand, eval, and stats
- Using the Events, Patterns, Statistics, and Visualization views as stages of an investigation
- Correlating activity over time to distinguish automated attacks from isolated failures
- Integrating SIEM analysis with the manual and scripted workflows from Objectives 1 and 2

Incident Analysis — Characterising the SSH Brute-Force Campaign

The three objectives above followed the project brief. This section does not: it takes the output of all three stages and treats the dataset as an actual incident to be scoped, quantified, and explained, which is what a SOC analyst would be expected to produce after the tooling work is finished.

Each earlier stage established that something suspicious was present. None of them answered how much, how fast, from where, or against what. The figures below were derived from the raw log file and cross-checked against the CSV exports and the Splunk event counts.

Attack Summary

Metric	Value
Total log lines analysed	2,000
Suspicious entries identified	607 (30.4% of the file)
Authentication failures	490
Unknown-user probes	117
“Failed password” matches	0 — see Secondary Finding
Distinct attacking sources	47
Attempts against root	351 of 372 named-user attempts (94.4%)
Other accounts targeted	guest (17), test (4)
Observation window	41 days (14 June 15:16 – 26 July 07:04)
Busiest single source	150.183.249.110 — 80 attempts in 95 seconds
Fastest observed rate	220.117.241.87 — 13 attempts in 13 seconds (60/min)
Events indexed in Splunk	1,296
Events matching the detection filter	514

Top Attacking Sources

Forty-seven distinct sources generated the 607 suspicious entries, but the distribution is heavily skewed — the ten most active accounted for the majority of all attempts.

Source	Attempts	Target	Burst window	Rate
150.183.249.110	80	root	10 Jul 16:01:43 – 16:03:18 (95s)	50.5/min
n219076184117.netvigator.com	23	root	22 Jun 03:17:26 – 03:18:22 (56s)	24.6/min
207.243.167.114	23	root	26 Jul 07:02:27 – 07:04:12 (105s)	13.1/min
60.30.224.116	20	root	30 Jun (dispersed)	low and slow
195.129.24.210	15	root	30 Jun – 1 Jul (dispersed)	low and slow

Source	Attempts	Target	Burst window	Rate
218.188.2.4	14	(unknown user)	14 Jun – 15 Jun (dispersed)	low and slow
h64-187-1-131.gtconnect.net	13	root	29 Jun 12:11:53 – 12:12:10 (17s)	45.9/min
220.117.241.87	13	root	04 Jul 19:15:48 – 19:16:01 (13s)	60.0/min
220-135-151-1.hinet-ip.hinet.net	10	root	15 Jun 02:04:59 (single second)	burst
p15105218.pureserver.info	10	root	09 Jul 19:34:06 – 19:34:14 (8s)	75.0/min

Attack Timeline

The activity divides cleanly into two behavioural classes.

The first is the concentrated burst: a single source firing dozens of attempts within seconds, then stopping entirely. 150.183.249.110 produced 80 authentication failures against root in 95 seconds — roughly one attempt every 1.2 seconds — and never appeared again. 220.117.241.87 was faster still, at 13 attempts in 13 seconds. No human types at that rate; this is tooling working through a credential list against a host it has already identified as reachable.

The second is low-and-slow probing: sources such as 60.30.224.116 and 195.129.24.210 spread a comparable number of attempts across many hours. That pacing is a deliberate evasion technique — it stays beneath the per-minute thresholds that simple rate-based alerting relies on, which is precisely why detection built purely on “N failures in M minutes” misses it.

Both classes target the same account. Of 372 attempts that named a user, 351 targeted root — 94.4%. The remainder tried guest and test, the two other accounts most likely to exist with weak or default credentials.

Root Cause

The host named combo was running an SSH service reachable from the public internet with password authentication enabled and no evident rate limiting. That combination is sufficient on its own to attract this traffic. Nothing here indicates the host was specifically chosen or that the attackers had prior knowledge of it — the pattern of many unrelated sources arriving independently over a 41-day window is the signature of internet-wide scanning, in which automated tools enumerate reachable SSH endpoints and attempt common credentials against whatever they find.

The presence of “check pass; user unknown” alongside the failures confirms that usernames were being guessed as well as passwords, meaning the attackers had no prior knowledge of valid accounts on the system.

Remediation

The controls that would materially reduce this exposure, in rough order of effectiveness:

- Disable password authentication for SSH entirely and require key-based authentication — this removes the attack class rather than slowing it
- Disable direct root login (PermitRootLogin no), which invalidates 94.4% of the attempts observed here in a single configuration change
- Restrict SSH exposure to known networks with a firewall rule, VPN, or bastion host, removing the host from the population that internet-wide scanners can reach

- Deploy fail2ban or equivalent to block source addresses automatically after a threshold of failures
- Alert on authentication-failure rate rather than on individual events — one failure is routine, 80 in 95 seconds is an incident
- Investigate the 43 “logrotate: ALERT exited abnormally” entries, since a failing log-rotation service threatens the integrity of the very evidence this analysis depends on

Secondary Finding: A Detection Rule That Matched Nothing

The detection logic in Objective 2 tests for three patterns. One of them — “Failed password” — matched zero events across all 2,000 lines. The same string was included in the Splunk filter in Objective 3 and contributed nothing there either.

This is not a coding error. It is a log-format mismatch. “Failed password for root from …” is the message OpenSSH writes directly through sshd. This host authenticates through the PAM stack, which writes “authentication failure; logname= uid=0 … user=root” instead. Both describe the same event; only the wording differs. A rule written against one format silently returns nothing against the other.

Had this dataset consisted only of PAM-formatted events and the rule set contained only the “Failed password” pattern, the detection would have reported a clean result while 490 authentication failures passed through unnoticed — a false negative produced by a rule that appeared to be working correctly. The lesson generalises well beyond this lab: a detection rule that fires on nothing is indistinguishable, from the outside, from a quiet environment. Coverage has to be validated against the log format actually in use, not assumed from the rule’s wording.

SOC Relevance

This project exercises the Tier 1 analyst workflow end to end at three levels of tooling maturity: noticing an anomaly by eye, encoding that judgement into a repeatable rule, and then scaling it to a platform where the same data can be queried in ways the original rule never anticipated. The progression is deliberate — each stage exposes a limitation that motivates the next.

It also demonstrates the two failure modes that matter most in detection engineering. Rate-based rules miss low-and-slow activity because the attacker paces beneath the threshold. Keyword rules miss events written in an unexpected format because the string never matches. Both were present in this dataset, and both were only visible because the analysis was carried out three different ways and the results compared.

Summary

By completing this project, I learned to:

- Read and interpret raw Linux authentication logs, identifying failed logins, unknown-user probes, and abnormal service exits in the syslog message format.
- Structure unstructured log data into an analysable form, parsing 2,000 free-text lines into typed spreadsheet columns for filtering and pivoting.
- Automate detection with Python, classifying events by type and exporting the results to CSV for downstream review — reducing a manual task to a repeatable one.
- Ingest, index, and query data in Splunk Enterprise, configuring source types, hosts, and inputs, then writing SPL searches to isolate authentication failures.
- Extract fields and build statistics in SPL using rex, mvexpand, eval, and stats to turn raw event text into countable, sortable categories.
- Visualise attack patterns over time, using Splunk's Events, Patterns, Statistics, and Visualization views to move from individual records to a behavioural picture.
- Quantify an incident rather than describe it, establishing scope, rate, duration, and target concentration from the data itself.
- Recognise detection coverage gaps, demonstrated by a keyword rule that matched zero events because it was written for a different log format than the one under analysis.

Repository Contents

Path	Description
README.md	Project overview, findings, and embedded evidence — the GitHub landing page
docs/Linux_Log_Analysis_Report.docx	Full illustrated walkthrough with all screenshots
docs/Linux_Log_Analysis_Report.pdf	PDF export for in-browser preview
scripts/Log_Analysis_Automation.py	Detection script — classifies suspicious entries and exports CSV
scripts/log_to_excel.py	Parser — converts the raw log into structured spreadsheet columns
splunk/queries.spl	All Splunk searches used, including the field-extraction query
data/Linux_2k.log	Source dataset (LogHub)
output/Full_suspicious_logs.csv	607 flagged entries from the full file
output/suspicious_logs.csv	140 flagged entries from lines 200–500
output/Linux_2k.xlsx	Structured spreadsheet used for manual analysis
screenshots/	Evidence images referenced by the README

Dataset Attribution

Linux_2k.log is taken from the LogHub collection of real-world log datasets (https://github.com/logpai/loghub), published for log-analysis and anomaly-detection research. The source addresses that appear in this report are those recorded in that public dataset and are reproduced here as analytical evidence.